



**ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA**
Universidad de Córdoba



TRABAJO FIN DE GRADO
Grado en Ingeniería Informática

Diseño Modular e Incremental para la Navegación Autónoma Contextual en Sistemas Embebidos de Bajo Coste

Autor

Felipe López Castro Directores

**Joaquín Cuéllar Padilla
Dr. Fernando León García**

Curso 2025-2026

Manual de Código

Felipe López Castro

9 de febrero de 2026

Índice general

1. Manual de Código	3
1.1. Archivo install.sh	3
1.2. Capa 2: Abstracción de Hardware	5
1.2.1. Archivo move.py	5
1.2.2. Archivo RPIservo.py	8
1.2.3. Archivo robotLight.py	12
1.2.4. Archivo camera_opencv.py	15
1.3. Capa 3: Visión y Percepción	20
1.3.1. Archivo base_camera.py	20
1.4. Archivo cv_processor.py	24
1.4.1. Archivo vision_line.py	33
1.5. Capa 4: Lógica de control	48
1.5.1. Archivo functions.py	48
1.6. Capa 5: Aplicación	72
1.6.1. Archivo app.py	72
1.6.2. Archivo webServer.py	75
1.6.3. Archivo GUI.py (cliente)	82

Capítulo 1

Manual de Código

En esta parte del documento se va a detallar todo el código usado por el servidor y el cliente. El código está comentado para facilitar su comprensión y se aloja en el [Repositorio del Proyecto](#). Se espera que para el correcto entendimiento del código, el lector tenga conocimientos básicos de programación en Python.

Para desarrollar el código del proyecto se ha tomado como base el repositorio [adeept_picar-b](#) y ha sido modificado para adaptarlo a las necesidades del proyecto.

Se van a agrupar los archivos según la capa a la que pertenezcan:

1.1. Archivo install.sh

Este archivo se ha creado con el objetivo de instalar las librerías y dependencias necesarias para el correcto funcionamiento del resto de archivos.

```
1 #!/bin/bash
2 #
3 # This file is part of  Diseño Modular e Incremental para la
4 # Navegación Autónoma Contextual en Sistemas Embebidos de Bajo Coste
5 #
6 # Copyright 2026 Felipe López Castro <i12locaf@uco.es>
7 #
8 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
9 # en Sistemas Embebidos de Bajo Coste is free software: you can
10 #   redistribute it
11 # and/or modify it under the terms of the GNU General Public License
12 # as published by the Free Software Foundation, either version 3 of
13 #   the License,
14 # or (at your option) any later version.
15 #
16 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
17 # en Sistemas Embebidos de Bajo Coste is distributed in the hope that
18 #   it will be useful,
19 # but WITHOUT ANY WARRANTY; without even the implied warranty of
20 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
21 # GNU General Public License for more details.
22 #
23 # You should have received a copy of the GNU General Public License
```

```

21 # along with Diseño Modular e Incremental para la Navegación Autónoma
    Contextual
22 # en Sistemas Embebidos de Bajo Coste.
23 # If not, see <https://www.gnu.org/licenses/>.
24 #
25
26 # Stop on error
27 set -e
28
29 echo "Starting installation for Adept PiCar-B on Raspbian Bookworm..."
    "
30
31 # 1. Update system package lists
32 echo "Updating system package lists..."
33 sudo apt update
34
35 # 2. Install system dependencies (APT)
36 # python3-full: ensures venv is available
37 # python3-picamera2: essential for the camera on Bookworm (libcamera
    based)
38 # python3-opencv: optimized opencv for Pi
39 # libzbar0: required for pyzbar
40 # python3-rpi.gpio: system-level GPIO approach (though we also install
    in venv, this ensures system deps are there)
41 echo "Installing system dependencies..."
42 sudo apt install -y python3-full python3-picamera2 python3-opencv
    libzbar0 python3-tk i2c-tools python3-pip
43
44 # 3. Create Python Virtual Environment
45 # We use --system-site-packages to access python3-picamera2 and
    python3-opencv installed via apt
46 echo "Creating virtual environment (venv) with system site packages..."
    "
47 if [ -d "venv" ]; then
48     echo "Virtual environment 'venv' already exists. Skipping creation
        ."
49 else
50     python3 -m venv --system-site-packages venv
51     echo "Virtual environment created."
52 fi
53
54 # 4. Install Python dependencies inside venv
55 echo "Installing Python libraries from requirements.txt..."
56 source venv/bin/activate
57
58 # Upgrade pip just in case
59 pip install --upgrade pip
60
61 # Install requirements
62 if [ -f "requirements.txt" ]; then
63     pip install -r requirements.txt
64 else
65     echo "Warning: requirements.txt not found!"
66 fi
67
68 echo "Installation complete!"
69 echo "To run the project, activate the environment first:"
70 echo "source venv/bin/activate"

```

1.2. Capa 2: Abstracción de Hardware

1.2.1. Archivo move.py

```
1  #!/usr/bin/env python3
2  #
3  # This file is part of  Diseño Modular e Incremental para la
4  # Navegación Autónoma Contextual en Sistemas Embebidos de Bajo Coste
5  #
6  # Copyright 2026 Felipe López Castro <i12locaf@uco.es>
7  #
8  # Diseño Modular e Incremental para la Navegación Autónoma Contextual
9  # en Sistemas Embebidos de Bajo Coste is free software: you can
10 # redistribute it
11 # and/or modify it under the terms of the GNU General Public License
12 # as published by the Free Software Foundation, either version 3 of
13 # the License,
14 # or (at your option) any later version.
15 #
16 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
17 # en Sistemas Embebidos de Bajo Coste is distributed in the hope that
18 # it will be useful,
19 # but WITHOUT ANY WARRANTY; without even the implied warranty of
20 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
21 # GNU General Public License for more details.
22 #
23 # You should have received a copy of the GNU General Public License
24 # along with Diseño Modular e Incremental para la Navegación Autónoma
25 # Contextual
26 # en Sistemas Embebidos de Bajo Coste.
27 # If not, see <https://www.gnu.org/licenses/>.
28 #
29 # Nombre del archivo: move.py
30 # Descripción: Control del Motor DC (Canal B - Tracción Trasera)
31
32 import time
33 import RPi.GPIO as GPIO
34
35 # =====
36 # Configuración de Pines (BCM) - Motor B
37 # =====
38 # Ojo aquí: Si ves que el coche va al revés cuando le dices "adelante",
39 # solo tienes que cambiar los números de PIN_IN1 y PIN_IN2.
40 PIN_EN = 4 # Enable (PWM) -> Esto controla la potencia/velocidad
41 PIN_IN1 = 14 # Input 1
42 PIN_IN2 = 15 # Input 2
43
44 # Estado Global
45 _pwm = None
46 _current_speed = 70 # Velocidad por defecto (0-100)
47
48 def setup():
49     """Inicializa los pines GPIO y el PWM."""
50     global _pwm
```

```

48     GPIO.setmode(GPIO.BCM)
49     GPIO.setwarnings(False)
50
51     # Configurar pines como salida
52     GPIO.setup(PIN_EN, GPIO.OUT)
53     GPIO.setup(PIN_IN1, GPIO.OUT)
54     GPIO.setup(PIN_IN2, GPIO.OUT)
55
56     # Arrancamos el PWM a 1000Hz. Es el pulso que le da fuerza al
    motor.
57     try:
58         _pwm = GPIO.PWM(PIN_EN, 1000)
59         _pwm.start(0) # Empieza parado (ciclo 0)
60     except Exception as e:
61         print(f"[Move] Error iniciando PWM: {e}")
62
63 def speed_set(speed_value):
64     """Actualiza la variable global de velocidad."""
65     global _current_speed
66     try:
67         val = int(speed_value)
68         # Limitamos la velocidad (0 a 100)
69         _current_speed = max(0, min(100, val))
70     except ValueError:
71         pass
72     return _current_speed
73
74 def forward(speed=None):
75     """Mueve el motor hacia adelante."""
76     global _current_speed
77
78     # Si no se pasa velocidad, usar la global
79     s = speed if speed is not None else _current_speed
80     s = max(0, min(100, int(s)))
81
82     # Pin_IN1 y Pin_IN2 determina la dirección hacia delante
83     GPIO.output(PIN_IN1, GPIO.LOW)
84     GPIO.output(PIN_IN2, GPIO.HIGH)
85
86     if _pwm:
87         _pwm.ChangeDutyCycle(s)
88
89 def backward(speed=None):
90     """Mueve el motor hacia atrás."""
91     global _current_speed
92
93     s = speed if speed is not None else _current_speed
94     s = max(0, min(100, int(s)))
95
96     # Invertimos la polaridad de los pines: ahora IN1 es HIGH y IN2 es
    LOW
97     GPIO.output(PIN_IN1, GPIO.HIGH)
98     GPIO.output(PIN_IN2, GPIO.LOW)
99
100    if _pwm:
101        _pwm.ChangeDutyCycle(s) # Indica la potencia a la que debe ir
    el motor
102

```

```

103 def stop():
104     """Detiene el motor."""
105     GPIO.output(PIN_IN1, GPIO.LOW)
106     GPIO.output(PIN_IN2, GPIO.LOW)
107     if _pwm:
108         _pwm.ChangeDutyCycle(0) # Potencia cero
109
110 # Alias para compatibilidad con webServer.py / functions.py
111 motorStop = stop
112
113 def destroy():
114     """Libera los recursos GPIO al cerrar."""
115     stop()
116     if _pwm:
117         try:
118             _pwm.stop()
119         except: pass
120     GPIO.cleanup() # Libera los pines GPIO
121
122 if __name__ == '__main__':
123     # Test simple si se ejecuta directamente
124     try:
125         setup()
126         print("Test: Adelante 50%")
127         forward(50)
128         time.sleep(2)
129         print("Test: Atrás 50%")
130         backward(50)
131         time.sleep(2)
132         stop()
133         destroy()
134     except KeyboardInterrupt:
135         destroy()

```


1.2.2. Archivo RPIservo.py

```
1 #!/usr/bin/env python3
2 #
3 # This file is part of Diseño Modular e Incremental para la
4 # Navegación Autónoma Contextual en Sistemas Embebidos de Bajo Coste
5 #
6 # Copyright 2026 Felipe López Castro <i12locaf@uco.es>
7 #
8 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
9 # en Sistemas Embebidos de Bajo Coste is free software: you can
10 # redistribute it
11 # and/or modify it under the terms of the GNU General Public License
12 # as published by the Free Software Foundation, either version 3 of
13 # the License,
14 # or (at your option) any later version.
15 #
16 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
17 # en Sistemas Embebidos de Bajo Coste is distributed in the hope that
18 # it will be useful,
19 # but WITHOUT ANY WARRANTY; without even the implied warranty of
20 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
21 # GNU General Public License for more details.
22 #
23 # You should have received a copy of the GNU General Public License
24 # along with Diseño Modular e Incremental para la Navegación Autónoma
25 # Contextual
26 # en Sistemas Embebidos de Bajo Coste.
27 # If not, see <https://www.gnu.org/licenses/>.
28 #
29
30 # Nombre del archivo: RPIservo.py
31 # Descripción: Módulo de control para servomotores utilizando el
32 # controlador PWM PCA9685.
33 # Este script gestiona la comunicación I2C y proporciona una interfaz
34 # de alto nivel para posicionar los servos.
35
36 import time
37 # Librerías de CircuitPython para acceso al hardware (I2C y pines).
38 import board
39 import busio
40 # Librerías de Adafruit para el controlador PCA9685 y gestión de
41 # servos.
42 from adafruit_pca9685 import PCA9685
43 from adafruit_motor import servo
44
45 # --- Estado Global del Módulo ---
46 pca = None # Instancia del controlador PCA9685.
47 servos = [] # Lista que almacena los objetos de control para
48 # cada canal de servo.
49 _initialized = False # Bandera para controlar la inicialización única
50 # del sistema.
51
52 def init():
53     """
54     Inicializa la comunicación I2C y configura el controlador PCA9685.
55     Esta función establece el bus I2C, configura la frecuencia PWM a
```

```

50Hz (estándar para servos)
48 e instancia los objetos de control para los 16 canales disponibles
    en la placa.
49 Si ocurre un error durante la inicialización, se captura la
    excepción y se deshabilita el control.
50 """
51 global pca, servos, _initialized
52
53 # Evitar reinicialización si ya está configurado.
54 if _initialized:
55     return
56
57 try:
58     # Inicialización del bus I2C utilizando los pines SDA y SCL
    definidos por la placa.
59     i2c = busio.I2C(board.SCL, board.SDA)
60
61     # Creación de la instancia del controlador PCA9685.
62     pca = PCA9685(i2c)
63     pca.frequency = 50 # Frecuencia de 50Hz requerida para servos
    analógicos y digitales estándar.
64
65     # Generación de los objetos Servo para los 16 canales (0-15).
66     # Los anchos de pulso (min_pulse=500, max_pulse=2500) están
    ajustados para servos comunes tipo SG90/MG90S.
67     servos = [servo.Servo(pca.channels[i], min_pulse=500,
    max_pulse=2500) for i in range(16)]
68
69     _initialized = True
70     print("[RPIServo] Controlador PCA9685 iniciado correctamente.")
71 )
72
73 except Exception as e:
74     print(f"[RPIServo] Error Crítico de Inicialización: No se
    detecta PCA9685. Detalles: {e}")
75     print("[RPIServo] Funcionalidad de servos deshabilitada.")
76     servos = [] #probar si sirve
77     _initialized = False
78
79 def move(servo_id, angle):
80     """
81     Mueve un servo específico a una posición angular determinada.
82
83     Parámetros:
84     - servo_id (int): Identificador del canal del servo (0-15).
85     - angle (float/int): Ángulo objetivo en grados (típicamente 0-180)
86     .
87
88     La función valida que el servo esté inicializado y que el ID sea v
    álido.
89     Además, restringe el ángulo dentro de los límites seguros (0-180)
    para evitar daños físicos.
90     """
91
92     if not _initialized or not servos:
93         return
94
95     if 0 <= servo_id <= 15:
96         # Restricción (clamping) del ángulo para asegurar que esté en

```

```

    el rango [0, 180].
94     val = max(0, min(180, angle))
95     try:
96         servos[servo_id].angle = val
97     except OSError:
98         # Captura de errores de comunicación I2C (ruido, timeouts)
99         # para evitar caída del programa.
100         pass
101
102 def stop(servo_id):
103     """
104     Desactiva la señal de control (PWM) para un servo específico.
105
106     Esto "libera" el servo, permitiendo que se mueva libremente si se
107     aplica fuerza externa,
108     y detiene el consumo de energía para mantener la posición.
109
110     Parámetros:
111     - servo_id (int): Identificador del canal del servo (0-15).
112     """
113     if not _initialized: return
114
115     if 0 <= servo_id <= 15:
116         try:
117             # Establecer el ángulo a None deshabilita la señal PWM en
118             # el canal especificado.
119             servos[servo_id].angle = None
120         except: pass
121
122 def cleanup():
123     """
124     Libera los recursos de hardware asociados al controlador PCA9685.
125     Se debe llamar al detener la aplicación para asegurar un cierre
126     limpio del bus I2C.
127     """
128     global pca
129     if pca:
130         try:
131             pca.deinit()
132         except: pass
133     print("[RPIServo] Recursos liberados y controlador finalizado.")
134
135 # --- Auto-inicialización ---
136 # Se intenta inicializar el módulo automáticamente al ser importado.
137 # Esto simplifica el uso en otros scripts, asegurando que el hardware
138 # esté listo.
139
140 init()
141
142 if __name__ == '__main__':
143     """
144     Bloque de prueba unitaria.
145     Ejecuta una secuencia de movimientos en los primeros tres servos
146     para verificar el funcionamiento.
147     """
148     print("Iniciando prueba de diagnóstico de servos (Canales 0-2)...")
149 )
150 time.sleep(1)
151 for s in [0, 1, 2]:

```

```
144     print(f"Probando Servo ID: {s}")
145     move(s, 90) # Posición central
146     time.sleep(0.5)
147     move(s, 60) # Izquierda / Abajo
148     time.sleep(0.5)
149     move(s, 120) # Derecha / Arriba
150     time.sleep(0.5)
151     move(s, 90) # Retorno al centro
152 cleanup()
153 print("Prueba finalizada.")
```

1.2.3. Archivo robotLight.py

```
154 #!/usr/bin/env python3
155 #
156 # This file is part of Diseño Modular e Incremental para la
157 # Navegación Autónoma Contextual en Sistemas Embebidos de Bajo Coste
158 #
159 # Copyright 2026 Felipe López Castro <i12locaf@uco.es>
160 #
161 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
162 # en Sistemas Embebidos de Bajo Coste is free software: you can
163 # redistribute it
164 # and/or modify it under the terms of the GNU General Public License
165 # as published by the Free Software Foundation, either version 3 of
166 # the License,
167 # or (at your option) any later version.
168 #
169 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
170 # en Sistemas Embebidos de Bajo Coste is distributed in the hope that
171 # it will be useful,
172 # but WITHOUT ANY WARRANTY; without even the implied warranty of
173 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
174 # GNU General Public License for more details.
175 #
176 # You should have received a copy of the GNU General Public License
177 # along with Diseño Modular e Incremental para la Navegación Autónoma
178 # Contextual
179 # en Sistemas Embebidos de Bajo Coste.
180 # If not, see <https://www.gnu.org/licenses/>.
181 #
182 # Nombre del archivo: robotLight.py
183 # Descripción: Módulo de control para el sistema de iluminación
184 # frontal (LEDs RGB).
185 # Utiliza la librería 'gpiozero' para una gestión simplificada de los
186 # estados de los pines GPIO.
187
188 from gpiozero import RGBLED
189
190 class RobotLight:
191     """
192     Clase para el control de los LEDs RGB frontales del robot.
193
194     Esta clase abstrae la complejidad del manejo de pines GPIO,
195     permitiendo encender,
196     apagar y cambiar el color de los LEDs mediante métodos de alto
197     nivel.
198     """
199     def __init__(self):
200         """
201         Inicializa la instancia de RobotLight y configura los pines
202         GPIO.
203
204         Se definen dos objetos RGBLED (izquierdo y derecho) asignados
205         a pines específicos de la Raspberry Pi.
206         Incluye manejo de excepciones para evitar que fallos en el
207         hardware detengan la ejecución del programa.
208         """
209         self.led_izquierdo = None
```

```

199     self.led_derecho = None
200     try:
201         # Configuración de pines GPIO para los LEDs (Esquema BCM).
202         # LED Izquierdo: Rojo=22, Verde=23, Azul=24
203         self.led_izquierdo = RGBLED(red=22, green=23, blue=24)
204         # LED Derecho: Rojo=10, Verde=9, Azul=25
205         self.led_derecho = RGBLED(red=10, green=9, blue=25)
206         print("[RobotLight] LEDs delanteros inicializados
correctamente.")
207     except Exception as e:
208         print(f"[RobotLight] Advertencia: Fallo en inicialización
de LEDs: {e}")
209
210     def front_all_off(self):
211         """
212         Apaga ambos LEDs delanteros inmediatamente.
213         """
214         if self.led_izquierdo: self.led_izquierdo.off()
215         if self.led_derecho: self.led_derecho.off()
216
217     def front_color(self, color_name):
218         """
219         Establece un color específico para ambos LEDs delanteros.
220
221         Parámetros:
222         - color_name (str): Nombre del color deseado (ej. 'red', 'blue', 'white').
223
224         Nota: El mapa de colores utiliza valores (R, G, B).
225         Dependiendo de si los LEDs son
226         de ánodo o cátodo común, estos valores pueden requerir inversión
227         (0=Encendido vs 1=Encendido).
228         El mapa actual asume una lógica invertida (Active LOW) donde 0
229         enciende el componente.
230         """
231         color_map = {
232             'red': (0, 1, 1), 'blue': (1, 1, 0), 'green': (1, 0, 1),
233             'white': (0, 0, 0), 'black': (1, 1, 1), 'yellow': (0, 0,
1),
234             'cyan': (1, 0, 0), 'magenta': (0, 1, 0)
235         }
236
237         if color_name in color_map:
238             val = color_map[color_name]
239             if self.led_izquierdo: self.led_izquierdo.color = val
240             if self.led_derecho: self.led_derecho.color = val
241
242     # --- Métodos de Señalización (Alias) ---
243     def front_turn_left(self):
244         """Indica giro a la izquierda (actualmente color blanco está
tico)."""
245         self.front_color('cyan')
246
247     def front_turn_right(self):
248         """Indica giro a la derecha (actualmente color blanco estático)."""
249         self.front_color('magenta')

```

```

248     # --- Métodos de Compatibilidad ---
249     # Estos métodos se mantienen vacíos para preservar la
compatibilidad con llamadas
250     # de versiones anteriores del software o interfaces que esperan
estas funciones.
251     def start(self): pass
252     def breath(self, *args): pass
253     def police(self): pass
254     def rainbow(self): pass
255     def rear_set_color(self, *args): pass
256
257     def cleanup(self):
258         """
259         Libera los recursos de hardware y cierra las conexiones GPIO
de manera segura.
260         Debe ser llamado antes de terminar el programa.
261         """
262         self.front_all_off()
263         if self.led_izquierdo: self.led_izquierdo.close()
264         if self.led_derecho: self.led_derecho.close()

```

1.2.4. Archivo camera_opencv.py

```
1 #
2 # This file is part of Diseño Modular e Incremental para la
3 # Navegación Autónoma Contextual en Sistemas Embebidos de Bajo Coste
4 #
5 # Copyright 2026 Felipe López Castro <i12locaf@uco.es>
6 #
7 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
8 # en Sistemas Embebidos de Bajo Coste is free software: you can
9 # redistribute it
10 # and/or modify it under the terms of the GNU General Public License
11 # as published by the Free Software Foundation, either version 3 of
12 # the License,
13 # or (at your option) any later version.
14 #
15 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
16 # en Sistemas Embebidos de Bajo Coste is distributed in the hope that
17 # it will be useful,
18 # but WITHOUT ANY WARRANTY; without even the implied warranty of
19 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
20 # GNU General Public License for more details.
21 #
22 # You should have received a copy of the GNU General Public License
23 # along with Diseño Modular e Incremental para la Navegación Autónoma
24 # Contextual
25 # en Sistemas Embebidos de Bajo Coste.
26 # If not, see <https://www.gnu.org/licenses/>.
27 #
28 # Nombre del archivo: camera_opencv.py
29 # Descripción: Módulo de abstracción de hardware para la cámara del
30 # robot.
31 # Implementa el patrón Singleton y utiliza libcamera (a través de
32 # Picamera2) para la captura eficiente de video.
33 # Gestiona el ciclo completo de adquisición de imágenes,
34 # procesamiento y distribución en formato MJPEG.
35
36 import time
37 import threading
38 import cv2
39 from picamera2 import Picamera2
40
41 from base_camera import BaseCamera
42
43 class Camera(BaseCamera):
44     """
45     Controlador principal de la cámara.
46
47     Esta clase hereda de BaseCamera y añade la lógica específica para
48     interactuar con el hardware (Picamera2).
49     Características principales:
50     - Patrón Singleton: Garantiza una única instancia de control para
51     la cámara física.
52     - Integración con OpenCV: Facilita la manipulación de imágenes y
53     superposición de gráficos.
54     - Procesamiento Asíncrono: Delega algoritmos de visión artificial
```



```

47     a 'cv_processor'.
48     """
49     _instance = None
50     _lock = threading.Lock()
51
52     # Inicialización de atributos estáticos para seguridad entre hilos
53     _cv = None
54     _cv_overlays = True
55
56     def __new__(cls, *args, **kwargs):
57         """
58         Implementación del patrón Singleton.
59         Asegura que solo se cree una instancia de la clase Camera,
60         protegiendo el acceso concurrente con un Lock.
61         """
62         with cls._lock:
63             if cls._instance is None:
64                 cls._instance = super().__new__(cls)
65             return cls._instance
66
67     def __init__(self):
68         """
69         Constructor de la clase.
70         Inicializa la cámara física, configura la resolución y
71         framerate, y prepara las estructuras de control.
72         """
73         # Prevención de re-inicialización en patrón Singleton
74         if getattr(self, "_initialized", False):
75             return
76         self._initialized = True
77
78         self._jpeg_quality = 40          # Calidad de compresión JPEG
79         (0-100) para balancear ancho de banda/calidad.
80         self.modeSelect = "none"        # Modo de operación actual (ej. '
81         trackLine', 'none').
82         self._fps_last_t = time.time()
83         self._fps_counter = 0
84         self._fps_value = 0.0
85
86         # --- Configuración de Hardware (Picamera2) ---
87         # Se establece una resolución de trabajo de 640x480 a 60 FPS
88         para fluidez en tiempo real.
89         self._p2 = Picamera2()
90         cfg = self._p2.create_preview_configuration(
91             main={"size": (640, 480)},
92             controls={"FrameRate": 60.0},
93             buffer_count=4,
94         )
95         self._p2.configure(cfg)
96         self._p2.set_controls({"FrameRate": 60.0})
97         self._p2.start()
98
99         # Referencia al procesador de visión (Lazy Loading)
100         self._cv = None
101         self._cv_overlays = True
102
103         super(Camera, self).__init__()

```

```

99
100 # -----
101 # Gestión de Instancia y Accesos
102 # -----
103
104 @classmethod
105 def get_instance(cls):
106     """Devuelve la instancia única de la clase Camera."""
107     return cls()
108
109 @property
110 def cv_thread(self):
111     """
112     Propiedad para acceso al procesador de visión (CVProcessor).
113     Utiliza instanciación diferida (Lazy Initialization) para
114     evitar errores de importación circular y asegurar que se cree solo
115     cuando sea necesario.
116     """
117     if self._cv is None:
118         from cv_processor import CVProcessor
119         self._cv = CVProcessor()
120         self._cv.draw_overlays = self._cv_overlays
121         self._cv.mode = self.modeSelect
122     return self._cv
123
124 def modeselect(self, mode: str):
125     """
126     Cambia el modo de operación de la visión artificial.
127
128     Parámetros:
129     - mode (str): Identificador del modo (ej. 'trackLine', 'none')
130     .
131
132     Esta función configura el procesador de visión según el modo
133     solicitado,
134     activando o pausando el procesamiento según corresponda.
135     """
136     print(f"[Camera] Cambio de modo -> {mode}")
137     self.modeSelect = mode
138
139     if self.modeSelect == 'trackLine':
140         self.cv_thread.mode = 'trackLine'
141         self.cv_thread.img_to_process = None
142
143     elif self.modeSelect == 'none':
144         self.cv_thread.pause()
145
146 @classmethod
147 def modeselect_class(cls, mode: str):
148     """Método de clase conveniente para cambiar el modo desde
149     contextos estáticos."""
150     cls.get_instance().modeselect(mode)
151
152 # -----
153 # Generador de Frames (Bucle Principal)
154 # -----
155
156 @staticmethod

```

```

152     def frames():
153         """
154         Generador principal del flujo de video (Pipeline).
155
156         Este método estático se ejecuta en el hilo de fondo de
157         BaseCamera.
158         Etapas del Pipeline:
159         1. Captura: Obtiene el array de píxeles RGB desde el hardware
160           (libcamera).
161         2. Conversión: Transforma el espacio de color a BGR (estándar
162           OpenCV).
163         3. Procesamiento: Delega en CVProcessor la detección de
164           objetos o líneas.
165         4. Telemetría: Calcula FPS e inserta datos en pantalla (HUD).
166         5. Codificación: Comprime la imagen resultante a formato JPEG.
167         """
168         cam = Camera.get_instance()
169
170         # Aseguramiento de atributos en entorno concurrente (
171         # inicialización defensiva)
172         if not hasattr(cam, "_cv"):
173             cam._cv = None
174         if not hasattr(cam, "_cv_overlays"):
175             cam._cv_overlays = True
176
177         # Carga inicial del procesador de visión si no existe
178         if cam._cv is None:
179             from cv_processor import CVProcessor
180             cam._cv = CVProcessor()
181             cam._cv.draw_overlays = cam._cv_overlays
182             cam._cv.mode = cam.modeSelect
183
184         while True:
185             # 1. Captura de imagen (Hardware - Picamera2)
186             rgb = cam._p2.capture_array()
187             img = cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR)
188
189             # 2. Procesamiento de Visión (Delegado)
190             try:
191                 cam._cv.mode = cam.modeSelect
192                 img = cam._cv.draw_elements_on_frame(img)
193             except Exception:
194                 pass
195
196             # 3. Cálculo de métricas de rendimiento (FPS)
197             cam._fps_counter += 1
198             now = time.time()
199             if (now - cam._fps_last_t) >= 1.0:
200                 cam._fps_value = cam._fps_counter / (now - cam.
201                 _fps_last_t)
202                 cam._fps_counter = 0
203                 cam._fps_last_t = now
204
205             # 4. Superposición de Telemetría (HUD)
206             try:
207                 vs = cam._cv.get_vehicle_status() if (cam._cv is not
208                 None) else {}
209                 v_speed = int(vs.get('speed', 0))

```

```

203         v_steer = int(vs.get('steer', 88.5))
204     except Exception:
205         v_speed, v_steer = 0, 88.5
206
207     v_steer = max(60, min(130, v_steer)) # Limitación visual
del ángulo
208
209     try:
210         hud = f"FPS {cam._fps_value:.1f} Vel {v_speed:>3}
Giro {v_steer:>3}"
211         cv2.putText(
212             img, hud, (8, 20),
213             cv2.FONT_HERSHEY_SIMPLEX, 0.55, (30, 230, 30), 2,
cv2.LINE_AA,
214         )
215     except Exception:
216         pass
217
218     # 5. Codificación y Envío
219     # Conversión de la matriz de imagen a bytes JPEG
comprimidos.
220     ok, jpeg = cv2.imencode(
221         ".jpg", img, [int(cv2.IMWRITE_JPEG_QUALITY), cam.
_ jpeg_quality]
222     )
223     if not ok:
224         continue
225
226     # Generación del frame en formato MJPEG multipart.
227     yield (
228         b"--frame\r\n"
229         b"Content-Type: image/jpeg\r\n\r\n" + jpeg.tobytes() +
b"\r\n"
230     )
231
232     def close(self):
233         """Detiene la cámara y libera los recursos de hardware."""
234         try:
235             self._p2.stop()
236         except Exception:
237             pass

```

1.3. Capa 3: Visión y Percepción

1.3.1. Archivo base_camera.py

```
265 #!/usr/bin/env python3
266 #
267 # This file is part of Diseño Modular e Incremental para la
268 # Navegación Autónoma Contextual en Sistemas Embebidos de Bajo Coste
269 #
270 # Copyright 2026 Felipe López Castro <i12locaf@uco.es>
271 #
272 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
273 # en Sistemas Embebidos de Bajo Coste is free software: you can
274 # redistribute it
275 # and/or modify it under the terms of the GNU General Public License
276 # as published by the Free Software Foundation, either version 3 of
277 # the License,
278 # or (at your option) any later version.
279 #
280 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
281 # en Sistemas Embebidos de Bajo Coste is distributed in the hope that
282 # it will be useful,
283 # but WITHOUT ANY WARRANTY; without even the implied warranty of
284 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
285 # GNU General Public License for more details.
286 #
287 # You should have received a copy of the GNU General Public License
288 # along with Diseño Modular e Incremental para la Navegación Autónoma
289 # Contextual
290 # en Sistemas Embebidos de Bajo Coste.
291 # If not, see <https://www.gnu.org/licenses/>.
292 #
293 # Nombre del archivo: base_camera.py
294 # Descripción: Implementación de una clase base abstracta para la
295 # gestión eficiente del streaming de video.
296 # Este módulo maneja la captura de frames en un hilo independiente y
297 # su distribución a múltiples clientes simultáneos.
298
299 import time
300 import threading
301 from threading import get_ident
302
303 class CameraEvent:
304     """
305     Gestor de eventos de sincronización para clientes de video.
306
307     Esta clase permite que múltiples clientes (hilos) esperen a que un
308     nuevo frame esté disponible.
309     Utiliza un diccionario para rastrear el estado de cada cliente
310     individualmente.
311     """
312     def __init__(self):
313         # Diccionario que almacena eventos por identificador de hilo (
314         thread ID).
315         self.events = {}
```

```

308     def wait(self):
309         """
310         Bloquea la ejecución del cliente actual hasta que se notifica
la llegada de un nuevo frame.
311
312         Si es la primera vez que el cliente solicita un frame, se crea
un nuevo evento de sincronización.
313         Retorna:
314             True si el evento fue activado, False si hubo un timeout (
dependiendo de la implementación de threading.Event).
315         """
316         ident = get_ident()
317         if ident not in self.events:
318             # Crea una nueva entrada para este cliente: [Objeto Evento
, Timestamp de última actividad]
319             self.events[ident] = [threading.Event(), time.time()]
320         return self.events[ident][0].wait()
321
322     def set(self):
323         """
324         Notifica a todos los clientes conectados que un nuevo frame
está listo para ser procesado.
325
326         Itera sobre todos los clientes registrados, activa sus eventos
y actualiza su timestamp.
327         También realiza una limpieza automática de clientes inactivos
(timeout > 5 segundos) para liberar recursos.
328         """
329         now = time.time()
330         remove = []
331         for ident, event in self.events.items():
332             if not event[0].is_set():
333                 # Si el evento no estaba activado, lo activamos y
actualizamos el tiempo.
334                 event[0].set()
335                 event[1] = now
336             else:
337                 # Si el evento ya estaba activado y ha pasado el
tiempo límite, se marca para eliminación.
338                 # Esto sucede si el cliente no ha consumido el evento
anterior (ha cerrado la conexión o es lento).
339                 if now - event[1] > 5:
340                     remove.append(ident)
341
342             # Eliminar clientes inactivos del registro
343             for ident in remove:
344                 del self.events[ident]
345
346     def clear(self):
347         """
348         Resetea el evento de señalización para el cliente actual.
349
350         Debe llamarse después de procesar un frame para prepararse
para la siguiente espera (wait).
351         """
352         ident = get_ident()
353         if ident in self.events:
354             self.events[ident][0].clear()

```

```

355
356 class BaseCamera:
357     """
358     Clase base abstracta para cámaras de streaming.
359
360     Implementa el patrón Singleton para el hilo de captura de fondo.
361     Esto asegura que solo haya un proceso capturando imágenes de la cá-
362     mara física, independientemente de cuántos clientes web estén
363     visualizando el stream.
364     """
365     thread = None          # Referencia al hilo de fondo único (
366     background thread)
367     frame = None          # Almacena el último frame capturado
368     last_access = 0       # Timestamp del último acceso de cualquier
369     cliente
370     event = CameraEvent() # Instancia compartida del gestor de eventos
371
372     def __init__(self):
373         """
374         Constructor de la clase.
375
376         Verifica si el hilo de captura está activo. Si no lo está,
377         inicia un nuevo hilo
378         que ejecutará el método '_thread'. Bloquea la inicialización
379         hasta que el primer
380         frame válido haya sido capturado para evitar errores en el
381         cliente.
382         """
383         if BaseCamera.thread is None:
384             BaseCamera.last_access = time.time()
385             # Inicio del hilo demonio (daemon) implícito al no hacer
386             join, encargado de la captura.
387             BaseCamera.thread = threading.Thread(target=self._thread)
388             BaseCamera.thread.start()
389
390             # Espera activa (bloqueo) hasta asegurar que hay datos de
391             imagen disponibles.
392             while self.get_frame() is None:
393                 time.sleep(0)
394
395     def get_frame(self):
396         """
397         Método principal para obtener la imagen actual.
398
399         Marca la actividad del cliente, espera la notificación de
400         nuevo frame (wait),
401         limpia la señal (clear) y retorna los datos de la imagen.
402         """
403         BaseCamera.last_access = time.time()
404
405         # Sincronización: esperar a que el hilo de fondo capture una
406         nueva imagen.
407         BaseCamera.event.wait()
408         BaseCamera.event.clear()
409
410         return BaseCamera.frame
411
412     @staticmethod

```

```

401     def frames():
402         """
403         Generador abstracto de frames.
404
405         Su responsabilidad es interactuar con el hardware (OpenCV,
406         PiCamera, etc.) y yield (ceder) los bytes de cada imagen capturada
407         .
408         """
409         raise RuntimeError('El método estático frames() debe ser
410         implementado por la subclase.')
411
412     @classmethod
413     def _thread(cls):
414         """
415         Lógica del hilo de fondo (Background Thread).
416
417         Obtiene el iterador del generador 'frames()' y entra en un
418         bucle continuo.
419         Para cada frame generado:
420         1. Actualiza la variable de clase 'frame'.
421         2. Notifica a todos los clientes esperando ('event.set()').
422
423         El hilo se detendrá automáticamente si no hay clientes activos
424         durante un periodo.
425         """
426         print('[BaseCamera] Hilo de captura iniciado.')
427         frames_iterator = cls.frames()
428         for frame in frames_iterator:
429             BaseCamera.frame = frame
430             BaseCamera.event.set()
431             time.sleep(0) # Cede el control brevemente para permitir
432             cambios de contexto.
433
434         BaseCamera.thread = None

```


1.4. Archivo cv_processor.py

```
1 #
2 # This file is part of  Diseño Modular e Incremental para la
3 # Navegación Autónoma Contextual en Sistemas Embebidos de Bajo Coste
4 #
5 # Copyright 2026 Felipe López Castro <i12locaf@uco.es>
6 #
7 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
8 # en Sistemas Embebidos de Bajo Coste is free software: you can
9 #   redistribute it
10 # and/or modify it under the terms of the GNU General Public License
11 # as published by the Free Software Foundation, either version 3 of
12 #   the License,
13 # or (at your option) any later version.
14 #
15 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
16 # en Sistemas Embebidos de Bajo Coste is distributed in the hope that
17 #   it will be useful,
18 # but WITHOUT ANY WARRANTY; without even the implied warranty of
19 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
20 # GNU General Public License for more details.
21 #
22 # You should have received a copy of the GNU General Public License
23 # along with Diseño Modular e Incremental para la Navegación Autónoma
24 # Contextual
25 # en Sistemas Embebidos de Bajo Coste.
26 # If not, see <https://www.gnu.org/licenses/>.
27 #
28 # Nombre del archivo: cv_processor.py
29 # Descripción: Módulo orquestador de visión artificial.
30 # Este componente implementa una arquitectura Productor-Consumidor
31 #   utilizando multiprocesamiento para separar
32 # la captura de video del análisis intensivo de imágenes (detección de
33 #   líneas).
34 # Además, gestiona un hilo dedicado para el escaneo de códigos QR,
35 #   asegurando un rendimiento fluido.
36 # Características:
37 # - Multiprocesamiento para evitar bloqueo por GIL (Global Interpreter
38 #   Lock).
39 # - Hilo secundario para lectura de códigos QR con la librería pyzbar.
40 # - Comunicación Inter-Procesos (IPC) mediante Colas.
41
42 import time
43 import threading
44 import multiprocessing as mp
45 from typing import Optional
46
47 import cv2
48 import numpy as np
49 from pyzbar import pyzbar
50
51 class CVProcessor:
52     """
53     Clase principal de procesamiento de visión (Controlador).
54     """
```

```

48     Actúa como interfaz entre la cámara y los algoritmos de análisis.
Recibe los fotogramas,
49     los submuestra y los envía a un proceso "worker" independiente
para detectar líneas.
50     Paralelamente, puede activar un escáner de códigos QR en segundo
plano.

51
52     Atributos:
53         mode (str): Modo de operación actual (ej. 'trackLine', 'none')
.
54         _line_state (dict): Diccionario compartido con el estado
actual de la detección (error, centro, etc.).
55         _vision_proc (Process): Proceso independiente encargado de
ejecutar 'vision_line.py'.
56     """
57     def __init__(self):
58         """
59         Constructor de la clase CVProcessor.
60         Inicializa las estructuras de datos, bloqueos de hilos (Locks)
y el proceso de trabajo (Worker).
61         """
62         self.mode: str = "none"
63         self.draw_overlays: bool = True
64         self.frame = None # Buffer para almacenar el frame actual
accesible por el hilo QR.
65         self._paused = False
66
67         # --- Estado Compartido de la Detección de Línea ---
68         # Almacena métricas críticas como el error de alineación (err)
y coordenadas (cx).
69         # Se protege con un Lock para evitar condiciones de carrera
entre el hilo que recibe datos y el que los lee.
70         self._line_state = {
71             "has_line": False, "err": 0, "cx": None,
72             "img_w": 0, "img_h": 0, "timestamp": 0.0,
73             # Datos específicos del subsistema QR
74             "qr_data": None,
75             "qr_color": None,
76             "qr_mode": None,
77             "qr_cycles": None,
78             "qr_valid": False,
79             "qr_timestamp": 0.0,
80         }
81         self._line_lock = threading.Lock()
82         self._line_seq = 0 # Secuencial para sincronización y detección
n de nuevos datos.
83
84         # --- Configuración del Escáner QR ---
85         self._qr_scanning = False
86         self._qr_scan_thread = None
87
88         # Filtros dinámicos de color (ignorados por el algoritmo de
visión)
89         self._ignore_colors = []
90         self._ignore_lock = threading.Lock()
91
92         # --- Multiprocesamiento ---
93         # Configuración del método de inicio del proceso. 'fork' es

```

```

preferible en Unix/Linux
# por su eficiencia en clonar el espacio de memoria (Copy-on-
Write).
start_method = "fork" if "fork" in mp.get_all_start_methods()
else "spawn"
ctx = mp.get_context(start_method)

# Colas de comunicación Inter-Procesos (IPC)
# _qin: Envía frames reducidos del proceso principal al worker
.
# _qout: Recibe tuplas (Estado, Overlay) del worker al proceso
principal.
self._qin = ctx.Queue(maxsize=1)
self._qout = ctx.Queue(maxsize=1)
self._algo_size = (320, 240) # Resolución optimizada
para procesamiento rápido.
self._every = 3 # Factor de submuestreo
temporal (procesar 1 de cada 3 frames).

self._vehicle_status = {'speed': 0, 'steer': 88.5}
self._status_lock = threading.Lock()

# Inicialización y arranque del Proceso de Visión
self._vision_proc = ctx.Process(
    target=_vision_worker_main,
    args=(self._qin, self._qout, self._algo_size),
    daemon=True, # El proceso morirá si el programa principal
termina.
)
self._vision_proc.start()

self._last_overlay: Optional[np.ndarray] = None
self._frame_i = 0 # Contador de frames para el submuestreo

# Optimización de OpenCV para desactivar su propio threading
interno y evitar conflictos.
try: cv2.setNumThreads(1)
except Exception: pass

# -----
# Métodos de Acceso al Estado (Thread-Safe)
# -----

def _publish_line_state(self, st: dict):
    """Actualiza el estado interno con los resultados recibidos
del worker."""
    with self._line_lock:
        self._line_state = st
        self._line_seq += 1

def get_line_state(self, wait_new: bool = False, last_seq:
Optional[int] = None,
                    timeout: float = 0.25):
    """
    Recupera el estado actual de la visión.

    Parámetros:
    - wait_new (bool): Si es True, bloquea hasta recibir un dato m

```

```

141     ás reciente que last_seq.
142         - last_seq (int): Último número de secuencia procesado por el
143           cliente.
144         - timeout (float): Tiempo máximo de espera en segundos.
145
146     Retorna:
147     - Tupla (estado, secuencia actual).
148     """
149     if wait_new and last_seq is not None:
150         t0 = time.time()
151         while (time.time() - t0) < timeout:
152             with self._line_lock:
153                 if self._line_seq != last_seq:
154                     break
155             time.sleep(0.004)
156     with self._line_lock:
157         return dict(self._line_state), self._line_seq
158
159     def set_vehicle_status(self, speed: int, steer: int):
160         """Actualiza el estado conocido del vehículo (telemetría)."""
161         with self._status_lock:
162             self._vehicle_status = {'speed': int(speed), 'steer': int(
163                 steer)}
164
165     def get_vehicle_status(self):
166         """Devuelve una copia del estado del vehículo."""
167         with self._status_lock:
168             return dict(self._vehicle_status)
169
170     # -----
171     # Control de Flujo del Procesamiento
172     # -----
173
174     def pause(self):
175         """Pausa temporalmente el envío de frames al worker."""
176         self._paused = True
177         print("[CVProcessor] Procesamiento pausado.")
178
179     def resume(self):
180         """Reanuda el procesamiento de visión."""
181         self._paused = False
182         print("[CVProcessor] Procesamiento reanudado.")
183
184     def set_ignore_colors(self, colors: list):
185         """Actualiza la lista de colores a ignorar durante la
186         segmentación."""
187         with self._ignore_lock:
188             self._ignore_colors = list(colors) if colors else []
189
190     def get_ignore_colors(self):
191         """Obtiene la lista actual de colores ignorados."""
192         with self._ignore_lock:
193             return list(self._ignore_colors)
194
195     # -----
196     # Sistema de Escaneo QR (Segundo Plano)
197     # -----

```

```

195     def start_qr_scan(self):
196         """Inicia el hilo dedicado a la detección de códigos QR."""
197         if self._qr_scanning:
198             return
199
200         self._qr_scanning = True
201         self._qr_scan_thread = threading.Thread(target=self.
202 _qr_scan_worker, daemon=True)
203         self._qr_scan_thread.start()
204         print("[CVProcessor] Iniciando escaneo QR.")
205
206     def stop_qr_scan(self):
207         """Detiene el hilo de escaneo QR."""
208         self._qr_scanning = False
209         print("[CVProcessor] Deteniendo escaneo QR.")
210
211     def _qr_scan_worker(self):
212         """
213         Lógica del hilo de decodificación QR.
214
215         Analiza periódicamente el frame actual almacenado ('self.frame
216 ') utilizando la librería pyzbar.
217         Si detecta un código válido con formato 'Color-Modo-Ciclos' (
218 ej. 'Yellow-A-3'), actualiza
219         el estado compartido y detiene el escaneo automáticamente.
220         """
221         print("[CVProcessor QR] Worker iniciado.")
222
223         while self._qr_scanning:
224             try:
225                 if self.frame is None:
226                     time.sleep(0.1)
227                     continue
228
229                 # Copia para evitar conflictos de lectura/escritura
230                 con el hilo principal
231                 frame = self.frame.copy()
232                 gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
233                 decoded_objects = pyzbar.decode(gray)
234
235                 if decoded_objects:
236                     for obj in decoded_objects:
237                         qr_data = obj.data.decode('utf-8').strip().
238 upper()
239
240                     # Validación del formato esperado: C-M-N
241                     parts = qr_data.split('-')
242                     if len(parts) == 3:
243                         color_code, mode_code, cycles_str = parts
244
245                     # Validaciones específicas de negocio
246                     valid_color = False
247                     if color_code == 'Y':
248                         color = 'yellow'; valid_color = True
249                     elif color_code == 'W':
250                         color = 'white'; valid_color = True
251
252                     valid_mode = mode_code in ['A', 'U']

```

```

248
249         try:
250             cycles = int(cycles_str)
251             valid_cycles = cycles > 0
252         except ValueError:
253             valid_cycles = False
254
255         if valid_color and valid_mode and
valid_cycles:
256             print(f"[CVProcessor QR] QR Válido
detectado: {color}, {mode_code}, {cycles}")
257
258             with self._line_lock:
259                 self._line_state['qr_data'] =
qr_data
260                 self._line_state['qr_color'] =
color
261                 self._line_state['qr_mode'] =
mode_code
262                 self._line_state['qr_cycles'] =
cycles
263                 self._line_state['qr_valid'] =
True
264                 self._line_state['qr_timestamp'] =
time.time()
265                 self._line_seq += 1
266
267                 self._qr_scanning = False
268                 return
269
270             time.sleep(0.1) # Tasa de refresco moderada para no
saturar CPU
271
272         except Exception as e:
273             print(f"[CVProcessor QR] Excepción en worker: {e}")
274             time.sleep(0.1)
275
276         print("[CVProcessor QR] Worker finalizado.")
277
278         # -----
279         # Integración con Pipeline de Cámara
280         # -----
281
282         def draw_elements_on_frame(self, frame_bgr: np.ndarray) -> np.
ndarray:
283             """
284             Método de orquestación llamado cíclicamente por el hilo de la
cámara.
285
286             1. Actualiza el buffer de imagen para el escáner QR.
287             2. Envía frames al proceso worker (aplicando submuestreo
temporal).
288             3. Recoge resultados asíncronos del worker.
289             4. Superpone gráficos de depuración (overlays) si están
habilitados.
290
291             Retorna:
292             El frame original (posiblemente modificado con

```

```

superposiciones).
293     """
294     self.frame = frame_bgr # Actualización buffer QR
295
296     if self.mode == "trackLine" and not self._paused:
297         self._frame_i += 1
298
299         # Submuestreo y Envío al proceso worker
300         if (self._frame_i % self._every) == 0:
301             try:
302                 small = cv2.resize(frame_bgr, self._algo_size,
interpolation=cv2.INTER_AREA)
303                 ignore_colors = self.get_ignore_colors()
304
305                 # Manejo de la cola llena: Descartes el frame más
antiguo si es necesario.
306                 if self._qin.full():
307                     try: self._qin.get_nowait()
308                     except Exception: pass
309
310                 self._qin.put_nowait((small, ignore_colors))
311             except Exception:
312                 pass
313
314             # Recogida de resultados (Non-blocking)
315             # Se intenta vaciar la cola para obtener siempre el
resultado más fresco disponible.
316             got = False
317             st = ov = None
318             while True:
319                 try:
320                     st, ov = self._qout.get_nowait()
321                     got = True
322                 except Exception:
323                     break
324
325             if got:
326                 self._publish_line_state(st)
327                 self._last_overlay = ov if self.draw_overlays else
None
328
329             else:
330                 self._last_overlay = None
331
332             # Renderizado de Overlay (Visualización de depuración)
333             if self._last_overlay is not None:
334                 try:
335                     # Escalar el overlay pequeño al tamaño original del
frame
336                     ov = cv2.resize(
337                         self._last_overlay,
338                         (frame_bgr.shape[1], frame_bgr.shape[0]),
339                         interpolation=cv2.INTER_LINEAR
340                     )
341                     frame_bgr = ov # Reemplazo directo (o superposición ló
gica según necesidad)
342                 except Exception:
343                     pass

```

```

344         return frame_bgr
345
346
347
348 # =====
349 # Función del Proceso Worker (Espacio de memoria separado)
350 # =====
351
352 def _vision_worker_main(qin: mp.Queue, qout: mp.Queue, algo_size:
353     tuple[int, int]):
354     """
355     Punto de entrada para el proceso secundario de visión.
356
357     Este bucle infinito se ejecuta en un núcleo separado. Consume imágenes, ejecuta
358     la lógica pesada de 'run_line_auto' (en vision_line.py) y devuelve los resultados.
359     Aísla los cálculos intensivos del hilo principal del servidor web.
360     """
361     try: cv2.setNumThreads(1)
362     except Exception: pass
363
364     from vision_line import run_line_auto
365
366     while True:
367         try:
368             # Espera bloqueante hasta recibir datos
369             data = qin.get()
370             if isinstance(data, tuple) and len(data) == 2:
371                 small, ignore_colors = data
372             else:
373                 small = data
374                 ignore_colors = []
375         except (EOFError, KeyboardInterrupt):
376             break # Terminación limpia
377         except Exception:
378             continue
379
380         try:
381             # Ejecución del algoritmo de visión
382             state, overlay = run_line_auto(small, draw_overlays=True,
383             ignore_colors=ignore_colors)
384         except Exception as e:
385             # Fallback seguro: Retornar estado vacío/neutro en caso de
386             error algorítmico,
387             # evitando que el proceso muera.
388             h, w = small.shape[:2]
389             state, overlay = ({
390                 "has_list": [False]*5, "cxs": [None]*5, "errs": [None]
391                 *5, "bands": [(0,0)]*5,
392                 "has_near": False, "err_near": 0, "cx_near": w//2, "
393                 band_near": (0,0),
394                 "has_mid": False, "err_mid": 0, "cx_mid": w//2, "
395                 band_mid": (0,0),
396                 "has_far": False, "err_far": 0, "cx_far": w//2, "
397                 band_far": (0,0),
398                 "has_line": False, "err": 0, "cx": w//2,
399                 "img_w": w, "img_h": h, "timestamp": time.time(),

```



```
393         "color_list": [None]*5,
394     }, None)
395
396     # Envío de resultados: Se vacía la cola de salida para
asegurar que solo
397     # se emite el resultado más reciente, evitando latencia
acumulada (backpressure).
398     try:
399         while True:
400             qout.get_nowait()
401         except Exception:
402             pass
403     qout.put((state, overlay))
```

1.4.1. Archivo vision_line.py

```
1 #
2 # This file is part of Diseño Modular e Incremental para la
3 # Navegación Autónoma Contextual en Sistemas Embebidos de Bajo Coste
4 #
5 # Copyright 2026 Felipe López Castro <i12locaf@uco.es>
6 #
7 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
8 # en Sistemas Embebidos de Bajo Coste is free software: you can
9 # redistribute it
10 # and/or modify it under the terms of the GNU General Public License
11 # as published by the Free Software Foundation, either version 3 of
12 # the License,
13 # or (at your option) any later version.
14 #
15 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
16 # en Sistemas Embebidos de Bajo Coste is distributed in the hope that
17 # it will be useful,
18 # but WITHOUT ANY WARRANTY; without even the implied warranty of
19 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
20 # GNU General Public License for more details.
21 #
22 # You should have received a copy of the GNU General Public License
23 # along with Diseño Modular e Incremental para la Navegación Autónoma
24 # Contextual
25 # en Sistemas Embebidos de Bajo Coste.
26 # If not, see <https://www.gnu.org/licenses/>.
27 #
28 #
29 # Nombre del archivo: vision_line.py
30 # Descripción: Módulo avanzado de visión artificial para la detección
31 # y seguimiento de líneas.
32 # Implementa un sistema de análisis por bandas horizontales para
33 # robustez algorítmica.
34 # Soporta múltiples colores (blanco, negro, rojo, amarillo) simultá
35 # neamente y utiliza
36 # una combinación de filtrado por color (HSV), umbralización
37 # adaptativa (Otsu) y análisis geométrico de contornos.
38
39 import cv2
40 import numpy as np
41 import time
42
43 # =====
44 # Perfiles de Color y Umbralización
45 # =====
46 # Definición de rangos HSV y estrategias de binarización para cada
47 # color de interés.
48 # Otsu Invertido se usa para objetos oscuros sobre fondo claro (línea
49 # negra).
50 # En el caso de que la iluminación cambie drásticamente, se debe
51 # ajustar los valores nuevamente.
52
53 BLACK_PROFILE = {
54     "hsv_lower": (0, 0, 0),
55     "hsv_upper": (179, 120, 255),
56     "otsu_invert": True,
```

```

46 }
47
48 WHITE_PROFILE = {
49     "hsv_lower": (88, 20, 240),
50     "hsv_upper": (113, 255, 255),
51     "otsu_invert": False,
52 }
53
54 RED_PROFILE = {
55     "hsv_lower": (0, 0, 0),
56     "hsv_upper": (179, 220, 255),
57     "otsu_invert": True,
58 }
59
60 YELLOW_PROFILE = {
61     "hsv_lower": (30, 30, 50),
62     "hsv_upper": (40, 170, 255),
63     "otsu_invert": False,
64 }
65
66 # =====
67 # Configuración del Sistema de Bandas (ROIs)
68 # =====
69 # La imagen se divide en 5 bandas horizontales para analizar la
    curvarura de la línea.
70 # Y_FRACS define la posición vertical relativa (0.0 a 1.0) de cada
    banda [inicio, fin].
71 Y_FRACS = [(0.00, 0), (0, 0), (0, 0), (0.25, 0.45), (0.45, 1.00)]
72 N_BANDS = 5
73 # BAND_ENABLED determina qué bandas se procesan activamente (por
    defecto, las inferiores).
74 BAND_ENABLED = [False, False, False, True, True]
75
76 # Parámetros de filtrado geométrico por banda
77 MIN_AREAS = [210, 240, 240, 300, 330] # Área mínima del contorno
    para ser considerado válido.
78 KERNEL_SIZES = [3, 3, 3, 3, 4] # Tamaño del kernel para
    operaciones morfológicas (erosión/dilatación).
79
80 # Parámetros de seguimiento ("Corridor Logic")
81 # Define el ancho esperado del camino para guiar la búsqueda en la
    siguiente banda.
82 CORRIDOR_HALVES = [30, 35, 40, 45, 65]
83 MAX_STEP_X = [30, 25, 22, 18, 9999] # Desplazamiento lateral máximo
    permitido entre frames consecutivos.
84
85 # Constantes de control
86 W_EMA = 0.6 # Peso del promedio móvil exponencial para
    suavizado de coordenadas (0.0 - 1.0).
87 WIDEN_STEP = 40 # Incremento del área de búsqueda si no se
    encuentra línea en la banda anterior.
88 MAX_WIDEN = 120 # Límite máximo de ampliación de búsqueda.
89 MAX_BAND_JUMP = 100 # Salto lateral máximo permitido entre bandas
    adyacentes (coherencia espacial).
90 FORCE_NEAR_VERTICAL = False
91 NEAR_LEN, NEAR_MIN_W, NEAR_MAX_W = 140, 10, 70
92
93 # Configuración Visual (Overlays)

```

```

94 BORDER = 3
95 COLORS = [(0, 0, 255), (0, 165, 255), (0, 255, 255), (0, 255, 165),
            (0, 255, 0)]
96
97 # Filtros de Calidad de Detección
98 FILL_RATIO_MAX = 0.32          # Máxima relación de llenado (área
    contorno / área banda) para evitar falsos positivos grandes.
99 ELONG_MIN_NON_NEAR = 1.4       # Elongación mínima requerida para
    considerar que un blob es una línea (y no una mancha circular).
100 CONTRAST_MIN_WHITE = 8.0      # Diferencia mínima de brillo (gris)
    entre línea blanca y fondo.
101 CONTRAST_MIN_BLACK = 10.0     # Diferencia mínima de brillo entre
    fondo y línea negra.
102 FILL_RATIO_MAX_WHITE = 0.45
103 FILL_RATIO_MAX_BLACK = 0.60
104
105 # Configuración de Texto en Pantalla (HUD)
106 TEXT_FONT_SCALE = 0.4
107 TEXT_THICKNESS = 1
108 TEXT_Y = 18
109 TEXT_COLOR = (50, 220, 50)
110
111 # Almacenamiento de estado entre frames (para suavizado)
112 _SW_PREV_CX = [None] * N_BANDS # Última posición X conocida por
    banda.
113 _POLARITY_PREV = [None] * N_BANDS # Último color detectado por banda.
114
115 # =====
116 # Configuración Visual ROW_MODE
117 # =====
118 ROW_MODE = True
119 ROW_PAD = 4
120 ROW_BOX_W = 45
121 ROW_THICK = 2
122 ROW_USE_POLARITY_COLOR = True
123 ROW_FILLED = True
124 ROW_FILL_ALPHA = 0.25
125
126 def _clamp(v, lo, hi):
127     """Restringe un valor dentro de un rango [lo, hi]."""
128     return max(lo, min(hi, v))
129
130 def _fill_rect_alpha(dst, pt1, pt2, color, alpha=0.25):
131     """Dibuja un rectángulo con transparencia sobre la imagen destino.
    """
132     lay = dst.copy()
133     cv2.rectangle(lay, pt1, pt2, color, -1)
134     cv2.addWeighted(lay, alpha, dst, 1.0 - alpha, 0, dst)
135
136 def _row_color(i, polarity):
137     """Selecciona el color de visualización basado en la polaridad
    detectada (tipo de línea)."""
138     if ROW_USE_POLARITY_COLOR and polarity in ("black", "white", "red"
    , "yellow"):
139         return (0, 180, 255) if polarity == "black" else \
140             (200, 200, 200) if polarity == "white" else \
141             (0, 0, 255) if polarity == "red" else \
142             (0, 255, 255)

```

```

143     return COLORS[i]
144
145 def _draw_row_box(overlay, i, y1, y2, cx, img_w, color, box_w=None,
146     filled=False, alpha=0.25):
147     """Función auxiliar para dibujar las cajas de seguimiento en el
148     overlay."""
149     if box_w is None:
150         box_w = ROW_BOX_W
151     row_h = (y2 - y1)
152     box_h = max(8, row_h - 2 * ROW_PAD)
153     cy = (y1 + y2) // 2
154     half_w = int(box_w // 2)
155     xL = _clamp(int(cx - half_w), 0, img_w - 1)
156     xR = _clamp(int(cx + half_w), 0, img_w - 1)
157     yT = _clamp(int(cy - box_h // 2), y1, y2 - 1)
158     yB = _clamp(int(cy + box_h // 2), y1, y2 - 1)
159     if filled:
160         _fill_rect_alpha(overlay, (xL, yT), (xR, yB), color, alpha)
161     cv2.rectangle(overlay, (xL, yT), (xR, yB), color, ROW_THICK)
162
163 # =====
164 # Funciones de Máscara (Segmentación)
165 # =====
166
167 def _mask_white(roi_bgr, roi_hsv, roi_gray, ksize=3):
168     """Genera máscara binaria para líneas blancas utilizando rango HSV
169     ."""
170     lower = np.array(WHITE_PROFILE["hsv_lower"], np.uint8)
171     upper = np.array(WHITE_PROFILE["hsv_upper"], np.uint8)
172
173     m = cv2.inRange(roi_hsv, lower, upper)
174
175     # Limpieza morfológica (Open -> Close) para eliminar ruido y
176     cerrar huecos
177     ker = np.ones((ksize, ksize), np.uint8)
178     m = cv2.morphologyEx(m, cv2.MORPH_OPEN, ker, iterations=1)
179     m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, ker, iterations=1)
180     return m
181
182 def _mask_black(roi_bgr, roi_hsv, roi_gray, ksize=3):
183     """
184     Genera máscara binaria para líneas negras.
185     Combina múltiples técnicas para robustez: HSV + Otsu (Invertido) +
186     Threshold Adaptativo.
187     """
188     # 1. Filtrado HSV (Rango de grises oscuros)
189     m_hsv = cv2.inRange(
190         roi_hsv,
191         np.array((47, 152, 0), np.uint8),
192         np.array((179, 255, 170), np.uint8)
193     )
194     # 2. Binarización de Otsu sobre imagen suavizada
195     blur = cv2.GaussianBlur(roi_gray, (5, 5), 0)
196     m_otsu = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.
197     THRESH_OTSU)[1]
198
199     # 3. Umbralización Adaptativa (Gaussian) para manejar iluminación
200     variable

```

```

194     m_adapt = cv2.adaptiveThreshold(
195         roi_gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.
THRESH_BINARY_INV, 31, 5
196     )
197
198     # Intersección de las tres máscaras (AND) -> Solo píxeles que
cumplan todos los criterios
199     m = cv2.bitwise_and(m_hsv, cv2.bitwise_and(m_otsu, m_adapt))
200
201     ker = np.ones((ksize, ksize), np.uint8)
202     m = cv2.morphologyEx(m, cv2.MORPH_OPEN, ker, iterations=1)
203     m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, ker, iterations=1)
204     return m
205
206
207 def _mask_red(roi_bgr, roi_hsv, roi_gray, ksize=3):
208     """
209     Genera máscara binaria para líneas rojas.
210     Maneja el matiz rojo que se divide en dos rangos en el espectro
HSV (0-10 y 160-180).
211     """
212     # Rango Rojo Bajo (0-10)
213     lower1 = np.array([0, 80, 50])
214     upper1 = np.array([10, 255, 255])
215     m1 = cv2.inRange(roi_hsv, lower1, upper1)
216
217     # Rango Rojo Alto (160-180)
218     lower2 = np.array([160, 80, 50])
219     upper2 = np.array([180, 255, 255])
220     m2 = cv2.inRange(roi_hsv, lower2, upper2)
221
222     # Unión de rangos (OR)
223     m_color = cv2.bitwise_or(m1, m2)
224
225     # Limpieza
226     ker = np.ones((ksize, ksize), np.uint8)
227     m = cv2.morphologyEx(m_color, cv2.MORPH_OPEN, ker, iterations=1)
228     m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, ker, iterations=1)
229     return m
230
231 def _mask_yellow(roi_bgr, roi_hsv, roi_gray, ksize=3):
232     """Genera máscara binaria para líneas amarillas."""
233     lower = np.array([45, 32, 165])
234     upper = np.array([80, 110, 255])
235
236     m_hsv = cv2.inRange(roi_hsv, lower, upper)
237
238     ker = np.ones((ksize, ksize), np.uint8)
239     m = cv2.morphologyEx(m_hsv, cv2.MORPH_OPEN, ker, iterations=1)
240     m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, ker, iterations=1)
241
242     return m
243
244 def _mask_from_profile(roi_hsv, roi_gray, profile, ksize=3):
245     """Función genérica para crear máscaras basadas en perfiles de
configuración."""
246     lower = np.array(profile["hsv_lower"], np.uint8)
247     upper = np.array(profile["hsv_upper"], np.uint8)

```

```

248     m_hsv = cv2.inRange(roi_hsv, lower, upper)
249     blur = cv2.GaussianBlur(roi_gray, (5, 5), 0)
250     th_flag = cv2.THRESH_BINARY_INV if profile.get("otsu_invert",
False) else cv2.THRESH_BINARY
251     m_otsu = cv2.threshold(blur, 0, 255, th_flag + cv2.THRESH_OTSU)[1]
252     m = cv2.bitwise_or(m_hsv, m_otsu)
253     ker = np.ones((ksize, ksize), np.uint8)
254     m = cv2.morphologyEx(m, cv2.MORPH_OPEN, ker, iterations=1)
255     m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, ker, iterations=1)
256     return m
257
258 def _largest_contour(mask):
259     """Retorna el contorno más grande encontrado en la máscara binaria
. """
260     cnts = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.
CHAIN_APPROX_SIMPLE)
261     cnts = cnts[0] if len(cnts) == 2 else cnts[1]
262     if not cnts:
263         return None
264     return max(cnts, key=cv2.contourArea)
265
266 def _mean_gray_in_contour(gray_roi, contour):
267     """Calcula el valor medio de gris dentro de un contorno específico
. """
268     mask = np.zeros_like(gray_roi, dtype=np.uint8)
269     cv2.drawContours(mask, [contour], -1, 255, -1)
270     return float(cv2.mean(gray_roi, mask=mask)[0])
271
272 def _score_contour(contour, band_area, corridor_center=None, cx_rect=
None):
273     """
274     Asigna una puntuación (score) a un contorno candidato.
275     Factores: Tamaño relativo al área de la banda y centralidad (si
aplica lógica de corredor).
276     """
277     if contour is None:
278         return 0.0
279     area = float(cv2.contourArea(contour))
280     if area <= 0:
281         return 0.0
282     score = max(0.0, min(1.0, area / max(1.0, float(band_area))))
283
284     # Penalización por distancia al centro esperado (Logic de Corredor
)
285     if corridor_center is not None and cx_rect is not None:
286         dist = abs(float(cx_rect) - float(corridor_center))
287         score *= 1.0 / (1.0 + dist / 40.0)
288     return score
289
290 # =====
291 # Núcleo de Detección por Banda
292 # =====
293 def _detect_band_auto(idx, frame, hsv_full, gray_full, w, h,
corridor_center=None, corridor_half=None,
294 draw_overlays=True, overlay=None, ignore_colors=
295 None):
296     """
297     Analiza una banda horizontal específica en busca de líneas de

```

```

cualquier color (Multicolor).
298
299     Aplica concurrentemente la detección de Negro, Blanco, Rojo y
Amarillo,
300     y selecciona el mejor candidato basado en puntuación y
consistencia.
301
302     Args:
303         idx (int): Índice de la banda (0-4).
304         ignore_colors (list): Colores a excluir explícitamente del análisis.
305         """
306         if ignore_colors is None:
307             ignore_colors = []
308
309         # Definición de la Región de Interés (ROI) vertical
310         y1 = int(h * Y_FRACS[idx][0])
311         y2 = int(h * Y_FRACS[idx][1])
312         roi_bgr = frame[y1:y2, :]
313         roi_hsv = hsv_full[y1:y2, :]
314         roi_gray = gray_full[y1:y2, :]
315
316         k = KERNEL_SIZES[idx]
317
318         # Generación de máscaras para colores permitidos
319         m_white = _mask_white(roi_bgr, roi_hsv, roi_gray, ksize=k) if '
white' not in ignore_colors else np.zeros_like(roi_gray)
320         m_black = _mask_black(roi_bgr, roi_hsv, roi_gray, ksize=k) if '
black' not in ignore_colors else np.zeros_like(roi_gray)
321         m_red = _mask_red(roi_bgr, roi_hsv, roi_gray, ksize=k) if 'red'
not in ignore_colors else np.zeros_like(roi_gray)
322         m_yellow = _mask_yellow(roi_bgr, roi_hsv, roi_gray, ksize=k) if '
yellow' not in ignore_colors else np.zeros_like(roi_gray)
323
324         # Extracción de contornos principales
325         c_b = _largest_contour(m_black)
326         c_w = _largest_contour(m_white)
327         c_r = _largest_contour(m_red)
328         c_y = _largest_contour(m_yellow)
329
330         # Validación de Contraste (necesaria para diferenciar línea de
suelo similar)
331         bg_med = float(np.median(roi_gray)) if roi_gray.size else 0.0
332         mu_b = _mean_gray_in_contour(roi_gray, c_b) if c_b is not None
else None
333         mu_w = _mean_gray_in_contour(roi_gray, c_w) if c_w is not None
else None
334
335         # El negro debe ser significativamente más oscuro que el fondo
336         ok_b = (mu_b is not None) and ((bg_med - mu_b) >=
CONTRAST_MIN_BLACK)
337         # El blanco debe ser significativamente más brillante que el fondo
338         ok_w = (mu_w is not None) and ((mu_w - bg_med) >=
CONTRAST_MIN_WHITE)
339
340         if not ok_b: c_b = None
341         if not ok_w: c_w = None
342

```



```

343     def process_contour(c, is_near, is_black):
344         """Analiza geometría del contorno: Rectángulo mínimo, elongación y área."""
345         if c is None:
346             return None, None, None, None, 0.0, None, None
347         rect = cv2.minAreaRect(c)
348         (cx_r, cy_r), (rw, rh), ang = rect
349
350         w_rect, h_rect = (min(rw, rh), max(rw, rh))
351         elong = (h_rect + 1e-6) / (w_rect + 1e-6)
352
353         # Filtro de forma: Descartar si es muy cuadrado/circular (salvo banda cercana)
354         if not is_near and elong < ELONG_MIN_NON_NEAR:
355             return None, None, None, None, 0.0, None, None
356
357         # Cálculo de área de la banda efectiva (para ratio de llenado)
358         if corridor_center is not None and corridor_half is not None:
359             xL = max(0, int(corridor_center - corridor_half))
360             xR = min(w, int(corridor_center + corridor_half))
361             band_area = float(max(1, (y2 - y1) * (xR - xL)))
362         else:
363             band_area = float(max(1, (y2 - y1) * w))
364
365         area = float(cv2.contourArea(c))
366         limit = FILL_RATIO_MAX_BLACK if is_black else FILL_RATIO_MAX_WHITE
367
368         # Filtro de tamaño excesivo (evitar detectar toda la carretera como línea)
369         if (area / band_area) > limit:
370             return None, None, None, None, 0.0, None, None
371
372         box = cv2.boxPoints(rect).astype(int)
373         score = _score_contour(c, band_area, corridor_center, cx_r)
374         return rect, box, int(cx_r), int(cy_r), score, area, band_area
375
376     # Procesamiento de todos los candidatos
377     is_near_band = (idx == 4)
378     rect_b, box_b, cx_b, cy_b, score_b, area_b, _ = process_contour(
379         c_b, is_near=is_near_band, is_black=True)
380     rect_w, box_w, cx_w, cy_w, score_w, area_w, _ = process_contour(
381         c_w, is_near=is_near_band, is_black=False)
382     rect_r, box_r, cx_r, cy_r, score_r, area_r, _ = process_contour(
383         c_r, is_near=is_near_band, is_black=False)
384     rect_y, box_y, cx_y, cy_y, score_y, area_y, _ = process_contour(
385         c_y, is_near=is_near_band, is_black=False)
386
387     # --- Lógica de Selección de Mejor Candidato ---
388     prev = _POLARITY_PREV[idx]
389     cand = []
390     if rect_b is not None: cand.append(('black', score_b))
391     if rect_w is not None: cand.append(('white', score_w))
392     if rect_r is not None: cand.append(('red', score_r))
393     if rect_y is not None: cand.append(('yellow', score_y))
394
395     if not cand:
396         chosen = None

```

```

393     else:
394         # Elegir el que tenga mayor puntuación
395         chosen = max(cand, key=lambda t: t[1])[0]
396         # Histéresis: Si ya seguíamos un color, preferimos mantenerlo
397         # salvo que el nuevo sea mucho mejor
398         if prev is not None:
399             s_chosen = dict(cand).get(chosen, 0.0)
400             s_prev = dict(cand).get(prev, 0.0)
401             if s_prev > 0 and (s_chosen / max(1e-6, s_prev)) < 1.2:
402                 chosen = prev
403
404     _POLARITY_PREV[idx] = chosen # Actualizar memoria de estado
405
406     if chosen is None:
407         if draw_overlays and overlay is not None and not ROW_MODE:
408             cv2.rectangle(overlay, (0, y1), (w - 1, y2 - 1), COLORS[
409 idx], 1)
410         return (False, None, None, y1, y2, None, None, None)
411
412     # Asignación de variables según el ganador
413     if chosen == 'black':
414         rect, box, cx, cy, score = rect_b, box_b, cx_b, cy_b, score_b
415         color_draw = (0, 0, 0)
416     elif chosen == 'white':
417         rect, box, cx, cy, score = rect_w, box_w, cx_w, cy_w, score_w
418         color_draw = (255, 255, 255)
419     elif chosen == 'red':
420         rect, box, cx, cy, score = rect_r, box_r, cx_r, cy_r, score_r
421         color_draw = (0, 255, 255)
422     elif chosen == 'yellow':
423         rect, box, cx, cy, score = rect_y, box_y, cx_y, cy_y, score_y
424         color_draw = (0, 0, 255)
425
426     used_w = None if not FORCE_NEAR_VERTICAL or idx != 4 or rect is
427 None else min(rect[1])
428
429     if rect is None or box is None:
430         return (False, None, None, y1, y2, None, None, chosen)
431
432     # Ajuste de coordenadas al marco de referencia de la imagen
433     # completa
434     box[:, 1] += y1
435     cx_abs = int(cx)
436     cy_abs = int(cy) + y1
437
438     # Dibujado de resultado en overlay (Modo Contorno)
439     if draw_overlays and overlay is not None and not ROW_MODE:
440         cv2.drawContours(overlay, [box], 0, color_draw, BORDER)
441         cv2.circle(overlay, (cx_abs, cy_abs), 3, color_draw, -1)
442         cv2.rectangle(overlay, (0, y1), (w - 1, y2 - 1), COLORS[idx],
443 1)
444         if corredor_center is not None and corredor_half is not None:
445             xL = max(0, int(corredor_center - corredor_half))
446             xR = min(w - 1, int(corredor_center + corredor_half))
447             cv2.rectangle(overlay, (xL, y1), (xR, y2 - 1), (50, 220,
448 50), 1)
449
450     return (True, cx_abs, cy_abs, y1, y2, box, used_w, chosen)

```

```

445
446 # =====
447 # Función Principal AUTO
448 # =====
449 def run_line_auto(frame: np.ndarray, draw_overlays: bool = True,
450                  ignore_colors: list = None):
451     """
452     Algoritmo Principal de Detección Automática.
453
454     Ejecuta el pipeline completo de visión:
455     1. Preprocesamiento (Conversión BGR -> HSV/Gray).
456     2. Iteración por bandas (de abajo hacia arriba, más cercana a más
457     lejano).
458     3. Mantenimiento del "corredor" (predicción de dónde debe estar la
459     línea en la siguiente banda).
460     4. Suavizado temporal de las coordenadas detectadas.
461     5. Generación de estructura de datos de estado y overlays visuales
462     .
463
464     Args:
465     frame: Imagen de entrada BGR.
466     draw_overlays: Booleano para activar graficos de debug.
467     ignore_colors: Lista de colores a excluir del análisis.
468
469     Returns:
470     Tupla (diccionario_estado, frame_con_overlay).
471     """
472     if ignore_colors is None:
473         ignore_colors = []
474
475     h, w = frame.shape[:2]
476     gray_full = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
477     hsv_full = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
478     center_x = w // 2
479     overlay = frame.copy() if draw_overlays else None
480
481     # Dibujado de líneas guía (Grid)
482     if draw_overlays and overlay is not None and ROW_MODE:
483         for i in range(len(Y_FRACS)):
484             y1g = int(h * Y_FRACS[i][0])
485             cv2.line(overlay, (0, y1g), (w - 1, y1g), (70, 70, 70), 1)
486             cv2.line(overlay, (0, h - 1), (w - 1, h - 1), (70, 70, 70), 1)
487
488     results = [None] * N_BANDS
489     color_list = [None] * N_BANDS
490     corredor_center = None
491     widen = 0
492     # Orden de procesamiento: De abajo (Cercano, Index 4) hacia arriba
493     # (Lejano, Index 0)
494     order = [4, 3, 2, 1, 0]
495
496     for i in order:
497         y1 = int(h * Y_FRACS[i][0])
498         y2 = int(h * Y_FRACS[i][1])
499
500         if not BAND_ENABLED[i]:
501             results[i] = {'found': False, 'cx': None, 'cy': None, '
band': (y1, y2), 'w_used': None}

```

```

497         color_list[i] = None
498         continue
499
500     # Lógica de Corredor: Dónde esperamos encontrar la línea basá
ndonos en la banda anterior?
501     expected = corredor_center if corredor_center is not None else
_SW_PREV_CX[i]
502     half = CORRIDOR_HALVES[i]
503     half_eff = None
504     if half is not None:
505         # Ampliamos la búsqueda si fallamos en la banda previa (
widen)
506         half_eff = min(half + widen, (MAX_WIDEN if i != 4 else
half))
507
508     found, cx, cy, y1, y2, box, used_w, chosen = _detect_band_auto
(
509         i, frame, hsv_full, gray_full, w, h,
510         corredor_center=expected if half_eff is not None else None
,
511         corredor_half=half_eff,
512         draw_overlays=draw_overlays, overlay=overlay,
513         ignore_colors=ignore_colors
514     )
515
516     # Validación de coherencia espacial (Salto brusco)
517     if found and corredor_center is not None and cx is not None:
518         dist = abs(cx - corredor_center)
519         if dist > MAX_BAND_JUMP:
520             found = False # Descartamos por incoherencia física
521             chosen = None
522         color_list[i] = chosen
523
524     if not found:
525         results[i] = {'found': False, 'cx': None, 'cy': None, '
band': (y1, y2), 'w_used': None}
526         widen += WIDEN_STEP # Incrementamos área de búsqueda para
la siguiente banda
527         if draw_overlays and overlay is not None and ROW_MODE:
528             cx_draw = int(_SW_PREV_CX[i]) if _SW_PREV_CX[i] is not
None else (w // 2)
529             color = _row_color(i, None)
530             _draw_row_box(overlay, i, y1, y2, cx_draw, w, color,
531                 box_w=None, filled=ROW_FILLED, alpha=
ROW_FILL_ALPHA)
532             continue
533
534     # Suavizado de coordenadas (Low-Pass Filter) para evitar
jitter
535     if _SW_PREV_CX[i] is not None and cx is not None:
536         dx = cx - _SW_PREV_CX[i]
537         max_dx = MAX_STEP_X[i]
538         # Clamping de la velocidad de movimiento lateral máxima
539         if abs(dx) > max_dx:
540             cx = int(_SW_PREV_CX[i] + max_dx * (1 if dx > 0 else
-1))
541
542     if _SW_PREV_CX[i] is None:

```

```

543         _SW_PREV_CX[i] = float(cx)
544     else:
545         _SW_PREV_CX[i] = (1.0 - W_EMA) * float(_SW_PREV_CX[i]) +
W_EMA * float(cx)
546
547     results[i] = {'found': True, 'cx': int(_SW_PREV_CX[i]), 'cy':
cy, 'band': (y1, y2), 'w_used': used_w}
548     corredor_center = results[i]['cx'] # Actualizamos centro para
la siguiente banda (arriba)
549     widen = 0 # Reseteamos ampliación de búsqueda al encontrar lí
nea
550
551     # Visualización en modo Filas (Cajas)
552     if draw_overlays and overlay is not None and ROW_MODE:
553         cx_draw = results[i]['cx']
554         pol = color_list[i]
555         color = _row_color(i, pol)
556         dyn_w = None
557         if results[i]['w_used'] is not None and i == 4:
558             dyn_w = int(_clamp(results[i]['w_used'] * 1.2, 30, max
(40, (y2 - y1) * 0.9)))
559         _draw_row_box(overlay, i, y1, y2, cx_draw, w, color,
560                       box_w=dyn_w, filled=ROW_FILLED, alpha=
ROW_FILL_ALPHA)
561
562     # Información HUD General
563     if draw_overlays and overlay is not None:
564         cv2.line(overlay, (center_x, 0), (center_x, h - 1), (0, 255,
255), 1)
565         status = " ".join([
566             f"{i}:{'B' if color_list[i] == 'black' else ('W' if
color_list[i] == 'white' else ('R' if color_list[i] == 'red' else
('Y' if color_list[i] == 'yellow' else '-')))}"
567             for i in range(N_BANDS - 1, -1, -1)
568         ])
569         cv2.putText(overlay, f"lineAuto | {status}", (10, TEXT_Y),
570                   cv2.FONT_HERSHEY_SIMPLEX, TEXT_FONT_SCALE,
TEXT_COLOR, TEXT_THICKNESS, cv2.LINE_AA)
571
572     # Compilación de resultados finales para el controlador
573     has_list = [r['found'] for r in results]
574     cx_list = [r['cx'] for r in results]
575     bands = [r['band'] for r in results]
576     err_list = [(center_x - cx) if cx is not None else None for cx in
cx_list]
577
578     state = {
579         'has_list': has_list,
580         'cxs': cx_list,
581         'errs': err_list,
582         'bands': bands,
583         'has_near': has_list[4], 'err_near': err_list[4], 'cx_near':
cx_list[4], 'band_near': bands[4],
584         'has_mid': has_list[2], 'err_mid': err_list[2], 'cx_mid':
cx_list[2], 'band_mid': bands[2],
585         'has_far': has_list[1], 'err_far': err_list[1], 'cx_far':
cx_list[1], 'band_far': bands[1],
586         'has_line': any(has_list),

```

```

587         'err': err_list[4], 'cx': cx_list[4], # Error principal basado
en banda cercana
588         'timestamp': time.time(), 'img_w': w, 'img_h': h,
589         'color_list': color_list,
590     }
591     return state, (overlay if draw_overlays else None)
592
593
594 # =====
595 # Wrappers de Compatibilidad
596 # =====
597 def _run_line_with_profile(frame: np.ndarray, profile: dict,
draw_overlays: bool = True):
598     """
599     Versión simplificada heredada para ejecutar detección con un solo
perfil de color.
600     Mantenido por compatibilidad hacia atrás con controladores
antiguos.
601     """
602     h, w = frame.shape[:2]
603     center_x = w // 2
604     overlay = frame.copy() if draw_overlays else None
605
606     hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
607     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
608
609     results = [None] * N_BANDS
610     corridor_center = None
611     widen = 0
612     order = [4, 3, 2, 1, 0]
613
614     for i in order:
615         y1 = int(h * Y_FRACS[i][0]); y2 = int(h * Y_FRACS[i][1])
616         if not BAND_ENABLED[i]:
617             results[i] = {'found': False, 'cx': None, 'cy': None, '
band': (y1, y2), 'w_used': None}
618             continue
619
620         roi_hsv = hsv[y1:y2, :]; roi_gray = gray[y1:y2, :]
621         k = KERNEL_SIZES[i]
622         mask = _mask_from_profile(roi_hsv, roi_gray, profile, ksize=k)
623         c = _largest_contour(mask)
624
625         if c is None:
626             results[i] = {'found': False, 'cx': None, 'cy': None, '
band': (y1, y2), 'w_used': None}
627             widen += WIDEN_STEP
628             if draw_overlays and overlay is not None and ROW_MODE:
629                 cx_draw = int(_SW_PREV_CX[i]) if _SW_PREV_CX[i] is not
None else (w // 2)
630                 color = _row_color(i, None)
631                 _draw_row_box(overlay, i, y1, y2, cx_draw, w, color,
box_w=None, filled=ROW_FILLED, alpha=
ROW_FILL_ALPHA)
632                 continue
633
634         rect = cv2.minAreaRect(c)
635         (cx_r, cy_r), (rw, rh), _ang = rect
636

```

```

637     w_rect, h_rect = (min(rw, rh), max(rw, rh))
638     elong = (h_rect + 1e-6) / (w_rect + 1e-6)
639     if i != 4 and elong < 1.4:
640         results[i] = {'found': False, 'cx': None, 'cy': None, '
641 band': (y1, y2), 'w_used': None}
642         widen += WIDEN_STEP
643         continue
644
645     cx_abs = int(cx_r)
646
647     if corridor_center is not None:
648         dist = abs(cx_abs - corridor_center)
649         if dist > MAX_BAND_JUMP:
650             results[i] = {'found': False, 'cx': None, 'cy': None,
651 'band': (y1, y2), 'w_used': None}
652             widen += WIDEN_STEP
653             continue
654
655     if _SW_PREV_CX[i] is not None:
656         dx = cx_abs - _SW_PREV_CX[i]
657         max_dx = MAX_STEP_X[i]
658         if abs(dx) > max_dx:
659             cx_abs = int(_SW_PREV_CX[i] + max_dx * (1 if dx > 0
660 else -1))
661         _SW_PREV_CX[i] = (1.0 - W_EMA) * float(_SW_PREV_CX[i]) +
662 W_EMA * float(cx_abs)
663     else:
664         _SW_PREV_CX[i] = float(cx_abs)
665
666     if FORCE_NEAR_VERTICAL and i == 4:
667         line_w = max(NEAR_MIN_W, min(NEAR_MAX_W, float(min(rw, rh)
668 )))
669         half_w = line_w / 2.0
670         half_L = float(NEAR_LEN) / 2.0
671         box = np.array([
672             [cx_r - half_w, cy_r - half_L],
673             [cx_r + half_w, cy_r - half_L],
674             [cx_r + half_w, cy_r + half_L],
675             [cx_r - half_w, cy_r + half_L],
676         ], dtype=np.float32).astype(int)
677     else:
678         box = cv2.boxPoints(rect).astype(int)
679
680     box[:, 1] += y1
681     cy_abs = int(cy_r) + y1
682
683     results[i] = {'found': True, 'cx': int(_SW_PREV_CX[i]), 'cy':
684 cy_abs, 'band': (y1, y2), 'w_used': None}
685     corridor_center = results[i]['cx']
686     widen = 0
687
688     if draw_overlays and overlay is not None:
689         if ROW_MODE:
690             cx_draw = results[i]['cx']
691             color = _row_color(i, None)
692             _draw_row_box(overlay, i, y1, y2, cx_draw, w, color,
693 box_w=None, filled=ROW_FILLED, alpha=

```

```

ROW_FILL_ALPHA)
689         else:
690             cv2.drawContours(overlay, [box], 0, (255, 255, 255),
BORDER)
691             cv2.circle(overlay, (int(_SW_PREV_CX[i]), cy_abs), 3,
(255, 255, 255), -1)
692             cv2.rectangle(overlay, (0, y1), (w - 1, y2 - 1),
COLORS[i], 1)
693
694     if draw_overlays and overlay is not None:
695         cv2.line(overlay, (center_x, 0), (center_x, h - 1), (0, 255,
255), 1)
696         if not ROW_MODE:
697             cv2.putText(overlay, "lineProfile", (10, TEXT_Y),
698                 cv2.FONT_HERSHEY_SIMPLEX, TEXT_FONT_SCALE,
TEXT_COLOR, TEXT_THICKNESS, cv2.LINE_AA)
699
700         has_list = [r['found'] for r in results]
701         cx_list = [r['cx'] for r in results]
702         bands = [r['band'] for r in results]
703         err_list = [(center_x - cx) if cx is not None else None for cx in
cx_list]
704
705         state = {
706             'has_list': has_list,
707             'cxs': cx_list,
708             'errs': err_list,
709             'bands': bands,
710             'has_near': has_list[4], 'err_near': err_list[4], 'cx_near':
cx_list[4], 'band_near': bands[4],
711             'has_mid': has_list[2], 'err_mid': err_list[2], 'cx_mid':
cx_list[2], 'band_mid': bands[2],
712             'has_far': has_list[1], 'err_far': err_list[1], 'cx_far':
cx_list[1], 'band_far': bands[1],
713             'has_line': any(has_list),
714             'err': err_list[4], 'cx': cx_list[4],
715             'timestamp': time.time(), 'img_w': w, 'img_h': h,
716         }
717         return state, (overlay if draw_overlays else None)
718
719 def run_line_black(frame: np.ndarray, draw_overlays: bool = True):
720     return _run_line_with_profile(frame, BLACK_PROFILE, draw_overlays)
721
722 def run_line_white(frame: np.ndarray, draw_overlays: bool = True):
723     return _run_line_with_profile(frame, WHITE_PROFILE, draw_overlays)
724
725 def run_line_red(frame: np.ndarray, draw_overlays: bool = True):
726     return _run_line_with_profile(frame, RED_PROFILE, draw_overlays)
727
728 def run_line_yellow(frame: np.ndarray, draw_overlays: bool = True):
729     return _run_line_with_profile(frame, YELLOW_PROFILE, draw_overlays)
)

```


1.5. Capa 4: Lógica de control

1.5.1. Archivo functions.py

```
1  #!/usr/bin/python3
2  #
3  # This file is part of  Diseño Modular e Incremental para la
4  # Navegación Autónoma Contextual en Sistemas Embebidos de Bajo Coste
5  #
6  # Copyright 2026 Felipe López Castro <i12locaf@uco.es>
7  #
8  # Diseño Modular e Incremental para la Navegación Autónoma Contextual
9  # en Sistemas Embebidos de Bajo Coste is free software: you can
10 # redistribute it
11 # and/or modify it under the terms of the GNU General Public License
12 # as published by the Free Software Foundation, either version 3 of
13 # the License,
14 # or (at your option) any later version.
15 #
16 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
17 # en Sistemas Embebidos de Bajo Coste is distributed in the hope that
18 # it will be useful,
19 # but WITHOUT ANY WARRANTY; without even the implied warranty of
20 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
21 # GNU General Public License for more details.
22 #
23 # You should have received a copy of the GNU General Public License
24 # along with Diseño Modular e Incremental para la Navegación Autónoma
25 # Contextual
26 # en Sistemas Embebidos de Bajo Coste.
27 # If not, see <https://www.gnu.org/licenses/>.
28 #
29 #
30 # Nombre del archivo: functions.py
31 # Descripción: Bucle de control principal (dirección + tracción)
32 # basado en visión artificial.
33 #
34 # Implementa lógica PID, máquinas de estado para manejo
35 # de QR,
36 #
37 # control de velocidad adaptativo y maniobras de
38 # recuperación.
39
40 import time
41 import threading
42
43 import move
44 import RPiServo
45 from camera_opencv import Camera
46 import numpy as np
47
48 # -----
49 # Constantes de Actuadores y Calibración
50 # -----
51
52 SERVO_TILT = 0      # Canal del servo de inclinación (Tilt)
53 SERVO_PAN  = 1      # Canal del servo de paneo (Pan)
54 SERVO_STEERING = 2  # Canal del servo de dirección
```

```

46 # --- Calibración de Dirección ---
47 # Valores PWM mapeados a ángulos físicos del servo de dirección.
48 STEER_RIGHT          = 50
49 STEER_CENTER         = 88.5    # Centro óptico/mecánico calibrado (
    ajustar si el coche deriva)
50 STEER_LEFT          = 120
51 # Ganancias del Controlador PID de Dirección
52 K_STEER              = 0.065   # Proporcional: Reacción directa al error
    (deg/px)
53 KD_STEER            = 0.15    # Derivativo: Amortiguación de
    oscilaciones
54 KI_STEER            = 0.01    # Integral: Corrección de error
    estacionario
55
56 # --- Configuración Servo Pan (Cámara) ---
57 # Seguimiento visual activo: la cámara gira en algunos casos para
    mantener la línea en el centro.
58 PAN_CENTER          = 85      # Posición central mirando al frente
59 PAN_MAX_LEFT        = 130     # Límite físico izquierdo
60 PAN_MAX_RIGHT       = 40      # Límite físico derecho
61
62 # Dinámica del seguimiento de cámara
63 PAN_GAIN            = 0.05    # Sensibilidad del movimiento de cámara al
    error
64 PAN_CENTER_FORCE    = 0.01    # "Resorte" virtual que devuelve la cámara
    al centro
65 PAN_MAX_STEP        = 6.0     # Máxima velocidad de giro por ciclo (
    grados)
66 PAN_DEADBAND        = 8       # Zona muerta en píxeles para evitar '
    jitter' (vibración)
67
68 # -----
69 # Constantes de Velocidad y Tracción
70 # -----
71 DRIVE_BASE_SPEED    = 60      # Velocidad cruce base (0-100)
72 DRIVE_MAX_SPEED     = 65      # Límite absoluto de velocidad automática
73 MIN_MOVE_SPEED      = 30      # PWM mínimo necesario para vencer la
    fricción estática
74
75 # --- Perfiles de Velocidad según Tipo de Línea ---
76 SPEED_BLACK_BOOST   = 80      # Velocidad alta para rectas negras (
    confianza alta)
77 SPEED_WHITE_NORMAL  = 55      # Velocidad moderada para terreno blanco
78 SPEED_YELLOW_MAX    = 45      # Velocidad reducida para zonas de
    precaución (amarillo)
79
80 # --- Parámetros de Búsqueda y Recuperación ---
81 COLOR_SEARCH_SPEED  = 55      # Velocidad durante maniobras de búsqueda
    de línea
82 YELLOW_CENTER_BIAS  = -5      # Desplazamiento intencional para seguir
    el carril amarillo
83 RED_STOP_TIME       = 2.0     # Tiempo de espera (s) al detectar señal
    de STOP (rojo)
84 RED_IGNORE_TIME     = 3.0     # Tiempo de gracia (s) después de un STOP
    para evitar falsos positivos
85
86 # --- Secuencias de Navegación (Misiones) ---
87 # Definen el orden esperado de colores en el circuito para navegación

```

```

    lógica.
88 WHITE_SEQUENCE = ["white", "black"]
89 YELLOW_SEQUENCE = ["yellow", "black", "yellow", "black"]
90 USE_YELLOW_SEQUENCE = True
91 USE_WHITE_SEQUENCE = True
92
93 # Maniobras de emergencia
94 SEARCH_TURN_SPEED      = 40    # Velocidad de giro
95 SEARCH_REVERSE_SPEED   = 50    # Velocidad de retroceso
96
97 # Ajustes específicos de dirección por color
98 STEER_GAIN_BLACK       = 0.15
99 BLACK_DEADBAND_PX      = 40    # Mayor tolerancia en línea negra para
    evitar correcciones nerviosas
100
101 # --- Reducción de Velocidad en Curvas ---
102 # El coche frena automáticamente si detecta que la línea se aleja del
    centro (curva cerrada)
103 BOTTOM_CENTER_SLOW_THRESH_PX = 110    # Umbral de error lateral para
    iniciar frenado
104 BOTTOM_CENTER_CUT_FRAC      = 0.10    # Porcentaje máximo de reducción
    de velocidad (10%)
105 BOTTOM_CENTER_DEADBAND_PX   = 8
106
107 # Umbrales críticos
108 ERR_SLOW_THRESH_PX    = 100    # Error para comenzar a reducir velocidad
    (general)
109 BLACK_SLOW_THRESH_PX  = 180    # Error tolerado en rectas
110 ERR_STOP_THRESH_PX    = 180    # Error crítico: Posible salida de pista
111
112 # Seguridad
113 NO_LINE_STOP_FRAMES   = 5      # Número de frames sin línea antes de
    detenerse/buscar
114
115 # --- Rampa de Aceleración (Soft Start/Stop) ---
116 RAMP_STEP_UP          = 10     # Incremento de velocidad por ciclo
117 RAMP_STEP_DOWN        = 10     # Decremento de velocidad por ciclo
118 RAMP_HZ_LIMIT         = 30.0   # Tasa máxima de actualización de motores
    (Hz)
119
120 # --- Ponderación de Ventanas de Visión ---
121 # La imagen se divide en 5 bandas. Los pesos determinan cuánto influye
    cada banda en la dirección.
122 # Pesos bajos arriba (lejos) y altos abajo (cerca).
123 WIN_WEIGHTS = [0.02, 0.03, 0.05, 0.20, 0.70]
124 PRED_GAIN      = 0.15    # Ganancia predictiva: Usa la pendiente de la lí
    nea para anticipar curvas
125
126 # Reducción anticipada de velocidad (Look-ahead)
127 MID_ERR_SLOW_THRESH = 150
128 FAR_ERR_SLOW_THRESH = 200
129 CUT_MID_FRAC = 0.05    # Reducción leve si la curva empieza a media
    distancia
130 CUT_FAR_FRAC = 0.05    # Reducción leve si la curva se ve a lo lejos
131
132 # --- Pre-giro (Feed-forward) ---
133 # Aplica un giro extra basado en la curvatura futura de la línea
134 FAR_PRESTEER_DEG = 0

```

```

135 MID_PRESTEER_DEG = 3
136 NEAR_PRESTEER_DEG = 10
137
138 # Búsqueda Activa (Active Search)
139 COLOR_SEARCH_SPEED_ACTIVE = 40
140 COLOR_SEARCH_STEER = 60
141
142 # Slew Rate: Limitador de velocidad del servo para movimientos más
    naturales
143 STEER_SLEW_DEG_PER_SEC = 60
144 STEER_SLEW_RATE_BLACK = 100 # Respuesta más rápida en carril rápido
145
146 FRESH_TIMEOUT_SEC = 1.0 # Tiempo máximo para considerar un dato de
    visión como válido
147
148 ANY_BAND_MIN_SPEED = 38
149
150 SEARCH_DEBOUNCE_FRAMES = 3 # Frames consecutivos necesarios para
    confirmar recuperación de línea
151 NEAR_EDGE_THRESH_PX = 100 # Umbral de borde para activar latch
    de búsqueda
152 SEARCH_FORWARD_TIMEOUT_S = 5.0 # Tiempo máximo intentando buscar
    hacia adelante antes de retroceder
153
154 # Constantes para maniobra de reversa
155 SEARCH_REACQUIRE_CENTER_PX = 40
156
157 # Pesos de respaldo (si falla la mezcla de ponderada)
158 NEAR_W, MID_W, FAR_W = 0.70, 0.20, 0.10
159
160 # --- Filtrado de Señal de Error ---
161 ERR_EMA_ALPHA = 0.50 # Factor de suavizado (Exponential Moving
    Average)
162 ERR_EMA_ALPHA_BLACK = 0.15 # Suavizado más agresivo en rectas
163 ERR_DEADBAND_PX = 6 # Ruido a ignorar
164 TANH_SCALE_PX = 140 # Escala para la función de modelado no
    lineal (tanh)
165 USE_TANH_SHAPING = True # Activa respuesta no lineal (más suave al
    centro, saturada en extremos)
166
167
168 class Functions(threading.Thread):
169     """
170     Hilo de CONTROL PRINCIPAL (Control Loop).
171
172     Orquesta la percepción y la acción:
173     1. Recibe el estado del entorno procesado por 'cv_processor'.
174     2. Ejecuta lógica de alto nivel (Misiones, QR, Recuperación).
175     3. Calcula comandos de bajo nivel (PID Dirección, Perfiles
    Velocidad).
176     4. Envía órdenes a los actuadores (Servos y Motores DC).
177     """
178
179     def __init__(self, *args, **kwargs):
180         """Inicialización de variables de estado y controladores."""
181         super(Functions, self).__init__(*args, **kwargs)
182
183         # Estado de modo de operación

```

```

184         self.functionMode = 'none'
185
186         # Variables de control de tracción
187         self.line_follow_active = False
188         self._target_speed = 0           # Velocidad deseada
189         self._current_speed = 0          # Velocidad actual (para rampa)
190         self._no_line_frames = 0         # Contador de pérdida de señal
visual
191         self._last_drive_time = 0.0
192         self._line_last_seq = None       # Sincronización con thread de
visión
193         self._last_debug_log = 0.0
194         self._ignore_red_until = 0.0     # Temporizador inercial tras
STOP
195
196         # Estado del servo de cámara (Pan)
197         self._pan_angle = float(PAN_CENTER)
198         self._pan_active = False
199
200         # Sincronización de hilos (Pause/Resume)
201         self.__flag = threading.Event()
202         self.__flag.clear()
203
204         # Estado del servo de dirección
205         self._servo_last_angle = STEER_CENTER
206         self._servo_last_time = time.time()
207
208         # --- Variables Latch para Recuperación ---
209         # "Latch" memoriza el último estado válido conocido para saber
hacia dónde girar
210         # si se pierde la línea.
211         self.last_near_err = None
212         self.last_near_side = None
213         self.last_near_color = None
214         self.search_latch = None          # Estado actual de la
maniobra de búsqueda
215         self.search_debounce = 0
216
217         self._err_ema = None              # Estado del filtro EMA
218
219         # Estado del controlador PID
220         self._pid_last_err = 0.0
221         self._pid_integral = 0.0
222         self._pid_last_time = time.time()
223
224         # Estado de secuencia de colores (Lógica de Misión)
225         self.white_sequence_index = 0
226         self.yellow_sequence_index = 0
227         self.target_color = None
228         self.active_sequence = None
229
230         # Sistema de lectura de códigos QR
231         self.qr_initial_color = None
232         self.qr_mode = None
233         self.qr_cycles_total = 0
234         self.qr_cycles_done = 0
235         self.qr_current_color = None
236         self.qr_needs_read = True

```

```

237
238     # Estado de búsqueda activa de color (Post-QR)
239     self.color_search_mode = False
240     self.color_search_target = None
241     self.color_search_direction = None
242     self.color_search_forced = False
243     self.color_search_frames = 0
244
245     # Variables para BARRIDO DE CÁMARA (Visual Sweep)
246     self.sweep_start_time = 0.0
247     self.sweep_direction = 1 # 1=Right, -1=Left
248
249     # Inicialización de hardware (Posición segura)
250     try:
251         RPIServo.move(SERVO_PAN, int(self._pan_angle))
252         RPIServo.move(SERVO_TILT, 40)
253     except:
254         pass
255
256     try:
257         if hasattr(move, "setup"):
258             move.setup()
259     except Exception as e:
260         print("[Functions] Error en inicialización de motores:", e
261 )
262
263     # -----
264     # Métodos Auxiliares de Movimiento y Control
265     # -----
266
267     def _update_telemetry(self, speed=None, steer=None):
268         """
269         Envía la telemetría actual (velocidad y dirección) al sistema
270         de cámara
271         para que se muestre en el HUD.
272         """
273         # Usar valores cacheados si no se especifican
274         s = int(self._current_speed) if speed is None else int(speed)
275         t = int(self._servo_last_angle) if steer is None else int(
276             steer)
277
278         try:
279             # Obtener instancia de cámara (Singleton) y actualizar
280             procesador
281             cam = Camera.get_instance()
282             if cam and cam.cv_thread:
283                 cam.cv_thread.set_vehicle_status(s, t)
284         except Exception:
285             pass # Evitar bloquear el hilo de control si la cámara
286             falla
287
288     def _update_pan_servo(self, error_px):
289         """
290         Sistema de "Mirada Activa" (Active Gaze).
291
292         Controla el servo Pan para mantener la línea centrada en la
293         imagen, independiente

```

```

289         del chasis del robot. Ayuda a no perder la línea en curvas
        cerradas.
290
291     Retorna:
292         El offset angular aplicado (útil para sumar a la dirección
        ).
293
294     """
295     if error_px is None:
296         # Retorno elástico al centro si no hay referencia visual
297         center_delta = (PAN_CENTER - self._pan_angle) * 0.1
298         self._pan_angle += center_delta
299     else:
300         # Corrección Proporcional basada en el error visual
301         vision_delta = 0
302         if abs(float(error_px)) > PAN_DEADBAND:
303             vision_delta = float(error_px) * PAN_GAIN
304
305         # Fuerza de centrado (para evitar que la cámara se quede
        girada permanentemente)
306         center_delta = (PAN_CENTER - self._pan_angle) *
        PAN_CENTER_FORCE
307
308         # Limitador de velocidad (Slew Rate Limiter) para
        movimiento suave
309         total_delta = vision_delta + center_delta
310         if total_delta > PAN_MAX_STEP: total_delta = PAN_MAX_STEP
311         elif total_delta < -PAN_MAX_STEP: total_delta = -
        PAN_MAX_STEP
312
313         self._pan_angle += total_delta
314
315         # Saturación a límites mecánicos
316         self._pan_angle = max(PAN_MAX_RIGHT, min(PAN_MAX_LEFT, self.
        _pan_angle))
317
318     try:
319         RPIServo.move(SERVO_PAN, int(self._pan_angle))
320     except:
321         pass
322
323     return (self._pan_angle - PAN_CENTER)
324
325 def _motor_stop(self):
326     """Detiene los motores DC inmediatamente."""
327     try:
328         if hasattr(move, "motorStop"):
329             move.motorStop()
330         else:
331             move.stop()
332     except Exception:
333         pass
334     self._current_speed = 0
335
336 def _set_target_speed(self, spd: int):
337     """Establece la consigna de velocidad objetivo."""
338     spd = int(max(0, min(100, spd)))
339     self._target_speed = spd

```

```

340 def _ramp_and_drive(self):
341     """
342     Controlador de Potencia de Motores con Rampa de Aceleración.
343
344     Suaviza la aceleración y frenado para:
345     1. Evitar picos de corriente que reinicien la Raspberry Pi.
346     2. Reducir el deslizamiento de las ruedas (tracción mecánica).
347     3. Proporcionar un movimiento más suave.
348     """
349     now = time.time()
350     # Limitador de frecuencia de actualización
351     if now - self._last_drive_time < 1.0 / RAMP_HZ_LIMIT:
352         return
353     self._last_drive_time = now
354
355     # Lógica de Rampa
356     if self._current_speed < self._target_speed:
357         # Aceleración
358         if self._current_speed < MIN_MOVE_SPEED:
359             self._current_speed = MIN_MOVE_SPEED # Kick-start
360         else:
361             self._current_speed = min(self._current_speed +
RAMP_STEP_UP, self._target_speed)
362
363     elif self._current_speed > self._target_speed:
364         # Frenado
365         self._current_speed = max(self._current_speed -
RAMP_STEP_DOWN, self._target_speed)
366
367     # Aplicación al hardware
368     v = int(self._current_speed)
369     print(f"[Functions] Vel: {v:3d} | Objetivo: {self.
_target_speed:3d}", end='\r', flush=True)
370
371     if (not self.line_follow_active) or (v < MIN_MOVE_SPEED and
self._target_speed == 0):
372         self._motor_stop()
373         self._update_telemetry(speed=0) # Actualizar telemetría a
0
374         return
375
376     try:
377         move.forward(v)
378         self._update_telemetry(speed=v) # Actualizar telemetría de
velocidad
379     except Exception:
380         self._motor_stop()
381
382 def _steer_from_err(self, err_px: int):
383     """Convierte error visual (px) directamente a ángulo de servo
(deg) [Modo Simple]."""
384     target = int(STEER_CENTER + K_STEER * int(err_px))
385     target = max(STEER_RIGHT, min(STEER_LEFT, target))
386     try:
387         RPiServo.move(SERVO_STEERING, target)
388     except Exception:
389         pass
390     return target

```



```

391
392     def _steer_command(self, target_deg: int, slew_rate=
STEER_SLEW_DEG_PER_SEC):
393         """
394         Envía comando al servo de dirección con limitación de
velocidad (Slew Rate).
395         Evita movimientos bruscos que puedan desestabilizar el vehí
culo.
396         """
397         now = time.time()
398         dt = max(1e-3, now - self._servo_last_time)
399         max_step = slew_rate * dt
400
401         # Saturación física
402         target_deg = max(STEER_RIGHT, min(STEER_LEFT, int(target_deg)))
)
403
404         # Limitación de paso
405         delta = target_deg - self._servo_last_angle
406         if abs(delta) > max_step:
407             target_deg = int(self._servo_last_angle + max_step * (1 if
delta > 0 else -1))
408
409         try:
410             RPIServo.move(SERVO_STEERING, target_deg)
411         except Exception:
412             pass
413
414         self._servo_last_angle = target_deg
415         self._servo_last_time = now
416
417         self._update_telemetry(steer=target_deg) # Actualizar telemetr
ía de dirección
418         return target_deg
419
420     def _filter_mix_err(self, err, alpha=ERR_EMA_ALPHA):
421         """
422         Preprocesamiento avanzado de la señal de error para el PID.
423         Pipeline: Deadband -> Filtro Paso Bajo (EMA) -> Conformado No
Lineal (Tanh).
424         """
425         if err is None:
426             return None
427
428         # 1. Zona muerta: Ignorar pequeños errores
429         if abs(err) < ERR_DEADBAND_PX:
430             err = 0.0
431
432         # 2. Filtro EMA (Media Móvil Exponencial) para reducir ruido
433         if self._err_ema is None:
434             self._err_ema = float(err)
435         else:
436             self._err_ema = (1.0 - alpha) * self._err_ema + alpha *
float(err)
437         e = self._err_ema
438
439         # 3. Función de transferencia no lineal (Tanh)
440         # Permite alta sensibilidad en el centro y satura suavemente

```

```

441     en los extremos
442         if USE_TANH_SHAPING and TANH_SCALE_PX > 1:
443             e = TANH_SCALE_PX * np.tanh(e / float(TANH_SCALE_PX))
444
445         return e
446
447     # -----
448     # Gestión de Modos y Estados
449     # -----
450
451     def pause(self):
452         """Transición a estado seguro: Motores OFF, Servos centrados.
453         """
454         self.functionMode = 'none'
455         self.line_follow_active = False
456
457         try:
458             self._set_target_speed(0)
459             self._ramp_and_drive()
460         except Exception:
461             try:
462                 if hasattr(move, "motorStop"): move.motorStop()
463                 else: move.stop()
464             except Exception: pass
465
466         self._target_speed = 0
467         self._current_speed = 0
468         self._no_line_frames = 0
469         self._line_last_seq = None
470
471         try: RPiServo.move(SERVO_STEERING, STEER_CENTER)
472         except Exception: pass
473
474         try:
475             # Desactiva procesamiento de visión para ahorrar CPU
476             cam = Camera.get_instance()
477             cam.modeselect('none')
478         except Exception: pass
479
480         try: self.__flag.clear()
481         except Exception: pass
482
483         print("[Functions] Sistema pausado.")
484
485     def modeSet(self, mode: str):
486         """Selector principal de modo de operación."""
487         if mode == 'trackLine':
488             self.trackLine()
489         elif mode == 'none':
490             self.pause()
491         else:
492             print(f"[Functions] Modo desconocido: {mode}")
493
494     def resume(self):
495         """Libera el bloqueo del thread principal."""
496         self.__flag.set()
497
498     def trackLine(self):

```

```

497     """
498     Inicializa y arranca el modo de Seguidor de Línea.
499     Resetea todos los controladores y estados internos para un
arranque limpio.
500     """
501     try:
502         if hasattr(move, "motorStop"): move.motorStop()
503         else: move.stop()
504     except Exception: pass
505
506     try: RPi servo.move(SERVO_STEERING, STEER_CENTER)
507     except Exception: pass
508
509     try:
510         cam = Camera.get_instance()
511         cam.modeselect('trackLine')
512         print("[trackLine] Visión activada.")
513     except Exception as e:
514         print(f"[trackLine] Error al activar visión: {e}")
515
516     # Reinicio de variables de control
517     self._target_speed = 0
518     self._current_speed = 0
519     self._no_line_frames = 0
520     self._line_last_seq = None
521     self._last_debug_log = 0.0
522     self.line_follow_active = True
523     self._ignore_red_until = 0.0
524
525     # Reinicio de variables de búsqueda (Latch)
526     self.last_near_err = None
527     self.last_near_side = None
528     self.last_near_color = None
529     self.search_latch = None
530     self.search_debounce = 0
531     self._err_ema = None
532
533     # Reinicio estado QR
534     self.qr_initial_color = None
535     self.qr_mode = None
536     self.qr_cycles_total = 0
537     self.qr_cycles_done = 0
538     self.qr_current_color = None
539     self.qr_needs_read = True
540
541     # Reinicio búsqueda activa
542     self.color_search_mode = False
543     self.color_search_target = None
544     self.color_search_direction = None
545     self.color_search_forced = False
546     self.color_search_frames = 0
547
548     # Reinicio Secuencias de color
549     self.white_sequence_index = 0
550     self.yellow_sequence_index = 0
551     self.target_color = None
552     self.active_sequence = None
553

```

```

554     # Reinicio PID
555     self._pid_last_err = 0.0
556     self._pid_integral = 0.0
557     self._pid_last_time = time.time()
558
559     self.functionMode = 'trackLine'
560     self.resume()
561
562     # Espera activa para estabilización de la cámara y primeros
frames válidos
563     try:
564         cvp = Camera.get_instance().cv_thread
565         t0 = time.time()
566         while time.time() - t0 < 0.5:
567             # Polling rápido para vaciar buffers viejos
568             st, seq = cvp.get_line_state(wait_new=True, last_seq=
None, timeout=0.2)
569             if (time.time() - st.get('timestamp', 0)) <
FRESH_TIMEOUT_SEC:
570                 break
571         except Exception:
572             pass
573
574     # -----
575     # Bucle Principal de Control (Llamado desde run)
576     # -----
577
578     def trackLineProcessing(self):
579         """
580         CICLO DE CONTROL CRÍTICO:
581         Ejecuta la lógica de conducción autónoma en tiempo real.
582
583         Etapas:
584         1. Adquisición de Datos: Obtiene geometría de líneas y colores
desde 'cv_processor'.
585         2. Gestión de Misión (QR): Decide qué color seguir y cuándo
detenerse.
586         3. Recuperación: Detecta si se perdió la línea e inicia
maniobras de búsqueda.
587         4. Navegación (Steering): Calcula el ángulo de giro usando PID
ponderado y Feed-forward.
588         5. Tracción (Throttle): Ajusta la velocidad según la curvatura
y condiciones de seguridad.
589         """
590         # 1. Adquisición de instancia de cámara
591         try:
592             cam = Camera.get_instance()
593             cvp = cam.cv_thread
594         except Exception:
595             time.sleep(0.1)
596         return
597
598         # 2. Sincronización bloqueante con el siguiente frame de visió
n
599         # Evita ejecutar el lazo PID más rápido que la cámara (ahorro
de CPU)
600         st, seq = cvp.get_line_state(wait_new=True, last_seq=self.
_line_last_seq, timeout=0.25)

```

```

601         self._line_last_seq = seq
602
603         # 3. Desempaquetado de datos del entorno
604         now = time.time()
605         fresh = (now - st.get('timestamp', 0)) < FRESH_TIMEOUT_SEC
606
607         errs = st.get('errs')          # Lista de errores desde arriba
608         (0) hacia abajo (4)
609         hasl = st.get('has_list')      # Banderas booleanas si se detect
610         ó línea en esa banda
611         en   = st.get('err_near', st.get('err', None)) # Error cercano
612         (fundamental para control)
613         em   = st.get('err_mid', None) # Error medio (anticipación)
614         ef   = st.get('err_far', None) # Error lejano (anticipación)
615         hn   = st.get('has_near', en is not None)
616         hm   = st.get('has_mid', em is not None)
617         hf   = st.get('has_far', ef is not None)
618
619         color_list = st.get('color_list', [None]*5)
620         raw_band_color = color_list[4] if (hasl and hasl[4]) else None
621
622         # -----
623         # Filtrado Lógico de Colores (QR / Misiones)
624         # -----
625         if self.qr_current_color is not None and not self.
626         qr_needs_read:
627             # Si tenemos una misión activa (ej. QR ordena seguir
628             AMARILLO):
629             # Filtramos todas las detecciones que NO sean de ese color
630             .
631
632             active_before = [(i, color_list[i]) for i in range(len(
633             hasl)) if hasl[i]]
634
635             filtered_hasl = list(hasl) if hasl else [False]*5
636             filtered_errs = list(errs) if errs else [None]*5
637             filtered_count = 0
638
639             for i in range(len(filtered_hasl)):
640                 if filtered_hasl[i]:
641                     band_color = color_list[i]
642
643                     # Máscara de exclusión lógica
644                     if self.qr_current_color == 'yellow':
645                         if band_color == 'white':
646                             filtered_hasl[i] = False
647                             filtered_errs[i] = None
648                             filtered_count += 1
649                     elif self.qr_current_color == 'white':
650                         if band_color == 'yellow':
651                             filtered_hasl[i] = False
652                             filtered_errs[i] = None
653                             filtered_count += 1
654
655             hasl = filtered_hasl
656             errs = filtered_errs
657
658         # --- Detección de pérdida de camino correcto ---

```

```

652         any_correct_color = any(has1)
653
654         if not any_correct_color and self.qr_current_color is not
None:
655             # No vemos el color correcto, Vemos el color
incorrecto? (ej. cruce o bifurcación)
656             wrong_color = 'white' if self.qr_current_color == '
yellow' else 'yellow'
657
658             sees_wrong_color = any(color_list[i] == wrong_color
for i in range(len(active_before))
659                                     if i < len(filtered_has1))
660
661             if sees_wrong_color:
662                 # Si vemos el color incorrecto, activamos el modo
de BÚSQUEDA DEL COLOR CORRECTO
663                 if not self.color_search_mode:
664                     print(f"[Color Search] Detectado {wrong_color.
upper()}, buscando {self.qr_current_color.upper()}")
665                     self.color_search_mode = True
666                     self.color_search_target = self.
qr_current_color
667                 else:
668                     # Recuperación: Si volvemos a ver el color correcto,
desactivamos búsqueda
669                     if self.color_search_mode:
670                         if self.color_search_forced:
671                             self.color_search_frames += 1
672                             if raw_band_color == self.color_search_target:
673                                 self.color_search_mode = False
674                                 self.color_search_target = None
675                                 self.color_search_direction = None
676                                 self.color_search_forced = False
677                                 self.color_search_frames = 0
678                             elif self.color_search_frames > 50: # Timeout
de búsqueda
679                                 self.color_search_mode = False
680                                 self.color_search_frames = 0
681                         else:
682                             self.color_search_mode = False
683                             self.color_search_target = None
684                             self.color_search_direction = None
685                             self.color_search_forced = False
686                             self.color_search_frames = 0
687
688                 # -----
689                 # Lógica de Secuencias Predefinidas (Fallback sin QR)
690                 # -----
691                 elif self.target_color is not None and (USE_WHITE_SEQUENCE or
USE_YELLOW_SEQUENCE):
692                     if self.active_sequence is None and raw_band_color in ['
white', 'yellow']:
693                         if raw_band_color == 'white' and USE_WHITE_SEQUENCE:
694                             self.active_sequence = 'white'
695                             self.target_color = WHITE_SEQUENCE[0]
696                         elif raw_band_color == 'yellow' and
USE_YELLOW_SEQUENCE:
697                             self.active_sequence = 'yellow'

```

```

698         self.target_color = YELLOW_SEQUENCE[0]
699
700         # Filtrado similar al modo QR
701         filtered_hasl = list(hasl) if hasl else [False]*5
702         for i in range(len(filtered_hasl)):
703             if filtered_hasl[i]:
704                 band_color = color_list[i]
705                 if self.active_sequence == 'white' and self.
target_color == 'white':
706                     if band_color == 'yellow':
707                         filtered_hasl[i] = False
708                     elif self.active_sequence == 'yellow' and self.
target_color == 'yellow':
709                         if band_color == 'white':
710                             RPiServo.move(SERVO_PAN, 70) # Ayuda
visual
711                             filtered_hasl[i] = False
712                             hasl = filtered_hasl
713
714         current_band_color = color_list[4] if (hasl and hasl[4]) else
None
715         any_band = isinstance(hasl, list) and any(hasl)
716
717         # Variables de banda auxiliar (índice 3) para robustez
718         en3 = errs[3] if (isinstance(errs, list) and len(errs) > 3)
else None
719         hn3 = hasl[3] if (isinstance(hasl, list) and len(hasl) > 3)
else False
720         color3 = color_list[3] if (isinstance(color_list, list) and
len(color_list) > 3) else None
721
722         # Actualización de memoria de última posición conocida (Latch)
723         if hn3 and en3 is not None:
724             self.last_near_side = 'left' if float(en3) > 0 else 'right
',
725
726             self.last_near_err = float(en3)
727             self.last_near_color = color3
728
729         # -----
730         # Lógica de Recuperación (State Machine)
731         # -----
732         if (self.search_latch is None) and (self.last_near_side in ('
left', 'right')):
733             cond_perdida = (not hn3) or (not any_band)
734             if cond_perdida:
735                 # Si se pierde la línea, decidimos la maniobra basada
en el último dato
736                 if (self.last_near_err is None) or (abs(self.
last_near_err) >= NEAR_EDGE_THRESH_PX):
737                     if self.last_near_color == 'yellow':
738                         self.search_latch = 'search_forward_right'
739                     elif self.last_near_side == 'left':
740                         # Si la línea se fue por la izquierda,
giramos izquierda buscándola
741                         self.search_latch = 'search_forward_left'
742                     else:
743                         self.search_latch = 'search_forward_right'

```

```

744         self.search_debounce = 0
745         self.search_started_t = now
746
747         # Ejecución de la máquina de estados de recuperación
748         if self.search_latch is not None:
749
750             # Condición de ÉXITO: recuperamos la línea centrada
751             if hn3 and en3 is not None and (abs(float(en3)) <
SEARCH_REACQUIRE_CENTER_PX):
752                 self.search_debounce += 1
753                 if self.search_debounce >= SEARCH_DEBOUNCE_FRAMES:
754                     self.search_latch = None
755                     self.search_debounce = 0
756                     self._motor_stop() # Breve parada para estabilizar
757
758             # Timeout de búsqueda frontal -> Pasar a búsqueda en
REVERSA
759             elif self.search_latch in ('search_forward_left', '
search_forward_right'):
760                 timeout_limit = SEARCH_FORWARD_TIMEOUT_S
761                 if (now - self.search_started_t) > timeout_limit:
762                     if self.search_latch == 'search_forward_left':
763                         self.search_latch = 'search_reverse_left'
764                     else:
765                         self.search_latch = 'search_reverse_right'
766
767                 # Avance forzado de secuencia si aplica
768                 if self.active_sequence == 'white' and
USE_WHITE_SEQUENCE:
769                     if self.white_sequence_index < len(
WHITE_SEQUENCE) - 1:
770                         self.white_sequence_index += 1
771                         self.target_color = WHITE_SEQUENCE[self.
white_sequence_index]
772                         self.search_latch = None
773                     elif self.active_sequence == 'yellow' and
USE_YELLOW_SEQUENCE:
774                     if self.yellow_sequence_index < len(
YELLOW_SEQUENCE) - 1:
775                         self.yellow_sequence_index += 1
776                         self.target_color = YELLOW_SEQUENCE[self.
yellow_sequence_index]
777                         self.search_latch = None
778
779             elif not hn3:
780                 self.search_debounce = 0
781
782             # Lógica Específica de BARRIDO DE CÁMARA
783             if self.search_latch == 'search_sweep':
784                 # Si hemos encontrado cualquier indicio de línea (
any_band), intentamos centrar y salir
785                 if any_band:
786                     # Dejar que el PID y Active Gaze hagan su trabajo
787                     self.search_latch = None
788                     self._motor_stop()
789                     return
790
791             # Ejecución del movimiento de barrido (Ping-Pong)

```



```

792         # Oscila entre PAN_MAX_LEFT (130) y PAN_MAX_RIGHT (40)
793         sweep_speed = 3.0 # Rad/s
794         dt_sweep = now - self.sweep_start_time
795
796         # Timeout del barrido: Si tarda mucho, pasamos a
MARCHA ATRÁS
797         if dt_sweep > 4.0: # 4 segundos de búsqueda
798             print("[Sweep] Barrido fallido. Iniciando
retroceso.")
799             self.search_latch = 'search_reverse_smart'
800             # Centrar cámara para retroceder
801             RPi servo.move(SERVO_PAN, PAN_CENTER)
802             self.sweep_start_time = now # Reusamos timer para
el reverso
803         else:
804             # Movimiento Sinusoidal para suavidad
805             # Centro 85, amplitud 45 -> [40, 130]
806             amplitude = 45
807             angle = PAN_CENTER + amplitude * np.sin(
sweep_speed * dt_sweep)
808             try:
809                 RPi servo.move(SERVO_PAN, int(angle))
810             except: pass
811
812             # Timeout para Reverse Smart
813             elif self.search_latch == 'search_reverse_smart':
814                 if (now - self.sweep_start_time) > 2.0: # 2 segundos
de marcha atrás
815                     # Si falla, volvemos a intentar barrido o paramos
?
816                     # Paramos para no irnos al infinito
817                     print("[Recovery] Recuperación fallida total.")
818                     self.search_latch = None
819                     self._motor_stop()
820
821         # -----
822         # 4. Cálculo de Dirección (Steering)
823         # -----
824         servo_base = None
825         mix_err = None
826
827         if self.color_search_mode:
828             # Modo búsqueda de color activa: forzar giro
829             if self.color_search_direction == 'right':
830                 servo_cmd = STEER_RIGHT
831             elif self.color_search_direction == 'left':
832                 servo_cmd = STEER_LEFT
833             else:
834                 servo_cmd = STEER_RIGHT
835
836             self._set_target_speed(COLOR_SEARCH_SPEED)
837             servo_pos = self._steer_command(servo_cmd)
838
839         elif self.search_latch is not None:
840             # Modo recuperación: giro predefinido según estado
841             if self.search_latch == 'search_forward_right':
842                 servo_cmd = STEER_RIGHT
843             elif self.search_latch == 'search_forward_left':

```

```

844         servo_cmd = STEER_LEFT
845     elif self.search_latch == 'search_reverse_right':
846         servo_cmd = STEER_LEFT # Invertido al retroceder
847     elif self.search_latch == 'search_reverse_left':
848         servo_cmd = STEER_RIGHT # Invertido al retroceder
849     else:
850         servo_cmd = STEER_CENTER
851
852
853     servo_pos = self._steer_command(servo_cmd)
854     else:
855         # Modo Seguimiento Normal: PID Ponderado
856         if isinstance(errs, list) and isinstance(hasl, list) and
len(errs) == len(hasl) == 5 and any_band:
857             # Calcular error promedio ponderado
858             acc = 0.0; wsum = 0.0
859             for i, e in enumerate(errs):
860                 if e is not None and hasl[i]:
861                     acc += WIN_WEIGHTS[i] * float(e)
862                     wsum += WIN_WEIGHTS[i]
863             if wsum > 0:
864                 mix_err = acc / wsum
865                 # Término D predictivo espacial (Gradiente visual)
866                 e_top = float(errs[0]) if (hasl[0] and errs[0]
is not None) else mix_err
867                 e_bottom = float(errs[4]) if (hasl[4] and errs[4]
is not None) else mix_err
868                 grad = e_bottom - e_top
869                 mix_err = mix_err + PRED_GAIN * grad
870
871                 # Filtrado temporal
872                 alpha_use = ERR_EMA_ALPHA_BLACK if (
current_band_color == 'black') else ERR_EMA_ALPHA
873                 mix_err = self._filter_mix_err(mix_err, alpha=
alpha_use)
874
875         if mix_err is None:
876             servo_base = STEER_CENTER
877         else:
878             now = time.time()
879             dt = max(1e-3, now - self._pid_last_time)
880
881             # --- CONTROLADOR PID ---
882             error = float(mix_err) # Proporcional
883
884             if current_band_color == 'black' and abs(error) <
BLACK_DEADBAND_PX:
885                 error = 0.0 # Zona muerta específica
886
887             # Integral (con limitador Anti-windup)
888             if not any_band:
889                 self._pid_integral = 0.0
890             else:
891                 self._pid_integral += error * dt
892                 self._pid_integral = max(-100.0, min(100.0, self.
_pid_integral))
893
894             # Derivativo (dError/dt)

```

```

895         derivative = (error - self._pid_last_err) / dt
896
897         self._pid_last_err = error
898         self._pid_last_time = now
899
900         # Salida del PID
901         pid_out = (K_STEER * error) + (KI_STEER * self._pid_integral) + (KD_STEER * derivative)
902
903         # Active Gaze: Mover cámara en curvas cerradas
904         pan_offset = 0
905         if current_band_color == 'yellow':
906             pan_offset = self._update_pan_servo(mix_err)
907             if pan_offset >= 75: pan_offset = 75
908         elif current_band_color == 'white':
909             pan_offset = self._update_pan_servo(mix_err)
910         else:
911             self._update_pan_servo(None) # Centrar cámara
912             pan_offset = 0
913
914         # Aplicación de salida
915         if current_band_color == 'black':
916             servo_base = int(STEER_CENTER + pid_out)
917             # Limitación suave para alta velocidad
918             servo_base = max(85.5, min(91.5, servo_base))
919         if current_band_color == 'yellow' or
current_band_color == 'white':
920             # Compensación de Pan en curvas lentas
921             servo_base = int(STEER_CENTER + pan_offset +
pid_out)
922         else:
923             servo_base = int(STEER_CENTER + pid_out)
924
925         # Pre-giro (Feed-forward) basado en visión lejana
926         pre_bias = 0
927         if hf and ef is not None:
928             pre_bias += int(np.sign(float(ef)) * FAR_PRESTEER_DEG)
929         if hm and em is not None:
930             pre_bias += int(np.sign(float(em)) * MID_PRESTEER_DEG)
931         if hn and en is not None:
932             pre_bias += int(np.sign(float(en)) * NEAR_PRESTEER_DEG
933     )
934
935     if servo_base is None:
936         servo_base = STEER_CENTER
937
938     servo_cmd = max(STEER_RIGHT, min(STEER_LEFT, servo_base +
pre_bias))
939
940     # Ajuste de velocidad del servo según urgencia
941     rate_use = STEER_SLEW_RATE_BLACK if (current_band_color ==
'black') else STEER_SLEW_DEG_PER_SEC
942     servo_pos = self._steer_command(servo_cmd, slew_rate=
rate_use)
943
944     # -----
945     # 5. Control de Velocidad (Throttle)
946     # -----

```

```

946     # Detección de STOP (Línea roja) y gestión de Misión QR
947     if current_band_color == 'red' and self.search_latch is None
948     and (time.time() > self._ignore_red_until):
949
950         # Caso A: Se requiere lectura de un QR
951         if self.qr_needs_read:
952             print("[QR] Deteniendo para escaneo de código...")
953             self._motor_stop()
954
955         try: # Pausar visión principal
956             cam = Camera.get_instance()
957             cvp = cam.cv_thread
958             cvp.pause()
959         except Exception: pass
960
961         # Posicionamiento de cámara para leer QR
962         try:
963             RPIservo.move(SERVO_TILT, 90) # Mirar arriba
964             RPIservo.move(SERVO_PAN, 85)
965             cvp.start_qr_scan() # Iniciar hilo scanner
966         except Exception: pass
967
968         # Espera bloqueante hasta leer código o timeout
969         qr_timeout = 60.0
970         qr_start_time = time.time()
971         qr_found = False
972
973         while (time.time() - qr_start_time) < qr_timeout:
974             try:
975                 st, seq = cvp.get_line_state(wait_new=False)
976                 if st.get('qr_valid', False):
977                     color = st.get('qr_color')
978                     mode = st.get('qr_mode')
979                     cycles = st.get('qr_cycles')
980                     qr_found = True
981                     print(f"[QR] Datos recibidos: Color={color
982 }, Modo={mode}, Ciclos={cycles}")
983                     break
984                 except Exception: pass
985                 time.sleep(0.1)
986
987             try: cvp.stop_qr_scan()
988             except: pass
989
990         # Configuración de la nueva misión
991         self.qr_initial_color = color
992         self.qr_mode = mode
993         self.qr_cycles_total = cycles
994         self.qr_cycles_done = 0
995         self.qr_current_color = color
996         self.qr_needs_read = False
997
998         # Configuración de búsqueda forzada para encontrar el
999         nuevo color objetivo
1000         self.color_search_mode = True
1001         self.color_search_target = color

```

```

1001         self.color_search_forced = True
1002         self.color_search_frames = 0
1003
1004         # Orientación inicial heurística según el color
objetivo
1005         if color == 'white':
1006             self.color_search_direction = 'left'
1007             RPIservo.move(SERVO_PAN, 100)
1008             try: cvp.set_ignore_colors(['yellow'])
1009                 except: pass
1010         elif color == 'yellow':
1011             self.color_search_direction = 'right'
1012             RPIservo.move(SERVO_PAN, 70)
1013             try: cvp.set_ignore_colors(['white'])
1014                 except: pass
1015         else:
1016             self.color_search_direction = 'left'
1017             try: cvp.set_ignore_colors([])
1018                 except: pass
1019
1020         try: cvp.resume()
1021         except: pass
1022
1023         # Restauración de actuadores para conducción
1024         try:
1025             RPIservo.move(SERVO_TILT, 40)
1026             RPIservo.move(SERVO_PAN, 85)
1027             RPIservo.move(SERVO_STEERING, 88.5)
1028         except: pass
1029
1030         # Limpiar flag de lectura
1031         try:
1032             with cvp._line_lock:
1033                 cvp._line_state['qr_valid'] = False
1034         except: pass
1035
1036         time.sleep(RED_STOP_TIME) # Espera semafórica
1037
1038         self._set_target_speed(50)
1039         self._ramp_and_drive()
1040         self._ignore_red_until = time.time() + RED_IGNORE_TIME
1041         return
1042
1043         # Caso B: Si ya tenemos misión, contamos ciclos (vueltas/
paradas)
1044         else:
1045             self.qr_cycles_done += 1
1046
1047         # Lógica Modo Único
1048         if self.qr_mode == 'U':
1049             if self.qr_cycles_done >= self.qr_cycles_total:
1050                 self.qr_needs_read = True # Misión cumplida,
esperar nuevo QR
1051                 print(["QR"] Misión completada.)
1052                 self._motor_stop()
1053                 return
1054             else:
1055                 self._ignore_red_until = time.time() +

```

```

1056         RED_IGNORE_TIME
1057         return
1058
1059         # Lógica Modo Alternado (Cambio de carril cada parada)
1060         elif self.qr_mode == 'A':
1061             total_stops = self.qr_cycles_total * 2
1062
1063             if self.qr_cycles_done >= total_stops:
1064                 self.qr_needs_read = True
1065                 self._motor_stop()
1066                 return
1067             else:
1068                 # Cambio de color objetivo (Toggle)
1069                 if self.qr_current_color == 'yellow':
1070                     self.qr_current_color = 'white'
1071                 else:
1072                     self.qr_current_color = 'yellow'
1073
1074                 try:
1075                     cam = Camera.get_instance()
1076                     cvp = cam.cv_thread
1077                     if self.qr_current_color == 'white':
1078                         cvp.set_ignore_colors(['yellow'])
1079                     elif self.qr_current_color == 'yellow':
1080                         cvp.set_ignore_colors(['white'])
1081                 except Exception: pass
1082
1083                 self._ignore_red_until = time.time() +
1084                 RED_IGNORE_TIME
1085
1086                 return
1087
1088                 # Aplicación de velocidad según estado de Navegación
1089                 if self.search_latch is not None:
1090                     # Velocidad reducida durante recuperación
1091                     if self.search_latch in ('search_forward_right', '
1092 search_forward_left'):
1093                         if self.last_near_color == 'yellow':
1094                             self._set_target_speed(SPEED_YELLOW_MAX)
1095                         else:
1096                             self._set_target_speed(SEARCH_TURN_SPEED)
1097                         self._ramp_and_drive()
1098
1099                     elif self.search_latch == 'search_sweep':
1100                         # Durante el barrido, mantenemos los motores detenidos
1101                         self._motor_stop()
1102
1103                     elif self.last_near_color == 'red' and self.
1104 qr_current_color == 'white':
1105                         try:
1106                             RPi servo.move(SERVO_PAN, PAN_MAX_LEFT)
1107                         except Exception:
1108                             pass
1109
1110                     else:
1111                         # Reverso (search_reverse_left/right/smart)
1112                         try:
1113                             move.backward(SEARCH_REVERSE_SPEED)
1114                             self._current_speed = -SEARCH_REVERSE_SPEED

```

```

1110         except Exception: self._motor_stop()
1111
1112     elif self.line_follow_active and fresh and any_band:
1113         # Conducción Normal: Determinar velocidad base
1114         is_yellow_any = False
1115         is_white_any = False
1116         if isinstance(color_list, list) and isinstance(hasl, list)
and len(color_list) == len(hasl):
1117             for i in range(len(color_list)):
1118                 if hasl[i]:
1119                     if color_list[i] == 'yellow': is_yellow_any =
True
1120                     if color_list[i] == 'white': is_white_any =
True
1121
1122             target_max = DRIVE_MAX_SPEED
1123
1124             # Prioridad de reducción de velocidad
1125             if is_yellow_any:
1126                 target_max = SPEED_YELLOW_MAX # Precaución
1127                 if self._current_speed > target_max:
1128                     self._current_speed = float(target_max)
1129             elif is_white_any:
1130                 target_max = SPEED_WHITE_NORMAL
1131             elif current_band_color == 'black':
1132                 target_max = SPEED_BLACK_BOOST
1133
1134             # Cálculo de frenado inteligente en curvas
1135             ref_err = float(en) if (hn and en) else (float(mix_err) if
mix_err else 0.0)
1136             abs_ref = abs(ref_err)
1137
1138             if abs_ref >= ERR_STOP_THRESH_PX:
1139                 # Curva muy crítica: Velocidad mínima
1140                 base = max(0, int(DRIVE_BASE_SPEED * 0.5))
1141             else:
1142                 # Interpolación lineal de velocidad basada en error
1143                 thresh_use = BLACK_SLOW_THRESH_PX if (
current_band_color == 'black') else ERR_SLOW_THRESH_PX
1144                 k = max(0.0, min(1.0, abs_ref / float(thresh_use)))
1145                 base = int(DRIVE_BASE_SPEED + (target_max -
DRIVE_BASE_SPEED) * (1.0 - k))
1146                 base = max(base, MIN_MOVE_SPEED)
1147
1148             # Reducciones adicionales predictivas
1149             cut_mid = cut_far = 0
1150             if hm and em is not None:
1151                 k_mid = max(0.0, min(1.0, abs(float(em)) / float(
MID_ERR_SLOW_THRESH)))
1152                 cut_mid = int(CUT_MID_FRAC * (DRIVE_MAX_SPEED -
MIN_MOVE_SPEED) * k_mid)
1153             if hf and ef is not None:
1154                 k_far = max(0.0, min(1.0, abs(float(ef)) / float(
FAR_ERR_SLOW_THRESH)))
1155                 cut_far = int(CUT_FAR_FRAC * (DRIVE_MAX_SPEED -
MIN_MOVE_SPEED) * k_far)
1156
1157             # Reducción por descentramiento acumulado

```

```

1158         errs_abs = []
1159         if isinstance(errs, list) and len(errs) >= 5:
1160             # Verificar desviaciones en la mitad inferior de la
imagen
1161             num_bands_to_check = len(errs) // 2
1162             start_index = len(errs) - num_bands_to_check
1163             for i in range(start_index, len(errs)):
1164                 if hasl[i] and errs[i] is not None:
1165                     off = abs(float(errs[i]))
1166                     off = max(0.0, off - float(
BOTTOM_CENTER_DEADBAND_PX))
1167                     errs_abs.append(off)
1168
1169             extra_cut = 0
1170             if errs_abs:
1171                 max_off = max(errs_abs)
1172                 k_center = max(0.0, min(1.0, max_off / float(
BOTTOM_CENTER_SLOW_THRESH_PX)))
1173                 extra_cut = int(BOTTOM_CENTER_CUT_FRAC * (
DRIVE_MAX_SPEED - MIN_MOVE_SPEED) * k_center)
1174
1175             desired = base - (cut_mid + cut_far + extra_cut)
1176
1177             if not hn:
1178                 desired = max(desired, ANY_BAND_MIN_SPEED)
1179
1180             self._no_line_frames = 0
1181             self._set_target_speed(int(max(MIN_MOVE_SPEED, desired)))
1182             self._ramp_and_drive()
1183
1184         else:
1185             # Parada de Seguridad por pérdida de señal sostenida
1186             self._no_line_frames += 1
1187             if self._no_line_frames >= NO_LINE_STOP_FRAMES:
1188                 self._motor_stop()
1189
1190             # Si no estamos ya en una maniobra, iniciamos BARRIDO
1191             #Pendiente de prueba
1192             if self.search_latch is None:
1193                 print("[Lost] Línea perdida. Iniciando barrido
visual...")
1194                 self.search_latch = 'search_sweep'
1195                 self.sweep_start_time = time.time()
1196
1197             # -----
1198             # Ejecución del Thread
1199             # -----
1200
1201         def functionGoing(self):
1202             if self.functionMode == 'trackLine':
1203                 self.trackLineProcessing()
1204
1205         def run(self):
1206             while True:
1207                 self.__flag.wait() # Espera hasta resume()
1208                 if self.functionMode != 'none':
1209                     self.functionGoing()

```


1.6. Capa 5: Aplicación

1.6.1. Archivo app.py

```
1  #!/usr/bin/env python3
2  #
3  # This file is part of  Diseño Modular e Incremental para la
4  # Navegación Autónoma Contextual en Sistemas Embebidos de Bajo Coste
5  #
6  # Copyright 2026 Felipe López Castro <i12locaf@uco.es>
7  #
8  # Diseño Modular e Incremental para la Navegación Autónoma Contextual
9  # en Sistemas Embebidos de Bajo Coste is free software: you can
10 # redistribute it
11 # and/or modify it under the terms of the GNU General Public License
12 # as published by the Free Software Foundation, either version 3 of
13 # the License,
14 # or (at your option) any later version.
15 #
16 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
17 # en Sistemas Embebidos de Bajo Coste is distributed in the hope that
18 # it will be useful,
19 # but WITHOUT ANY WARRANTY; without even the implied warranty of
20 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
21 # GNU General Public License for more details.
22 #
23 # You should have received a copy of the GNU General Public License
24 # along with Diseño Modular e Incremental para la Navegación Autónoma
25 # Contextual
26 # en Sistemas Embebidos de Bajo Coste.
27 # If not, see <https://www.gnu.org/licenses/>.
28 #
29 # Nombre del archivo: app.py
30 # Descripción: Este script implementa un servidor web basado en Flask
31 # diseñado para realizar la transmisión de video en tiempo real (
32 # streaming).
33
34 import os
35 # Importación de Flask y sus componentes para la gestión del servidor
36 # web, respuestas HTTP y envío de archivos.
37 from flask import Flask, Response, send_from_directory
38 # Importación de CORS para permitir solicitudes de recursos cruzados,
39 # facilitando la conexión desde clientes externos.
40 from flask_cors import CORS
41 # Importación de la clase Camera, responsable de la gestión del
42 # hardware de la cámara y la obtención de imágenes.
43 from camera_opencv import Camera
44
45 # Inicialización de la aplicación Flask.
46 app = Flask(__name__)
47 # Configuración de CORS para permitir credenciales y accesos desde
48 # distintos orígenes.
49 CORS(app, supports_credentials=True)
50
51 def gen():
```

```

43     """
44     Función generadora para el flujo de video.
45
46     Esta función se encarga de obtener continuamente los fotogramas
47     capturados por la cámara.
48     Instancia la clase Camera (que sigue el patrón Singleton para
49     asegurar una única instancia activa)
50     y formatea cada imagen como una parte de una respuesta multipart (
51     MIME type: multipart/x-mixed-replace).
52     Esto permite que el navegador o cliente actualice la imagen
53     constantemente, creando el efecto de video.
54     """
55
56     camera = Camera()
57     for frame in Camera.frames():
58         yield (b'--frame\r\n'
59               b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n')
60
61 @app.route('/video_feed')
62 def video_feed():
63     """
64     Ruta para el feed de video.
65
66     Endpoint: /video_feed
67     Método: GET
68
69     Esta función maneja las solicitudes a la ruta '/video_feed'.
70     Retorna una instancia de la clase Response de Flask
71     que encapsula el generador 'gen()'. Se define el tipo MIME como '
72     multipart/x-mixed-replace' con un límite 'frame',
73     lo cual es estándar para transmisiones de video MJPEG (Motion JPEG
74     ).
75     """
76     return Response(gen(), mimetype='multipart/x-mixed-replace;
77                     boundary=frame')
78
79 # --- Gestión de Rutas Estáticas y Página Principal ---
80 # Obtención de la ruta absoluta del directorio donde se encuentra este
81 # script.
82 dir_path = os.path.dirname(os.path.realpath(__file__))
83
84 @app.route('/')
85 def index():
86     """
87     Ruta raíz de la aplicación.
88
89     Endpoint: /
90     Método: GET
91
92     Devuelve un mensaje de texto simple indicando que el servidor de
93     video está operativo.
94     Sirve como comprobación básica de conectividad.
95     """
96     return "Servidor de Vídeo PiCar-B Activo. Para visualizar la
97     interfaz, utilice el cliente Python GUI.py."
98
99 if __name__ == '__main__':
100     """
101     Bloque principal de ejecución.

```

```
90
91     Inicia el servidor Flask si el script se ejecuta directamente.
92     Parámetros:
93     - host='0.0.0.0': Permite que el servidor sea accesible desde
94       cualquier dirección IP en la red local.
95     - port=5000: Puerto estándar para el servicio web.
96     - debug=False: El modo de depuración está desactivado para
97       producción.
98     - threaded=True: Habilita el manejo de múltiples solicitudes en
99       hilos separados para no bloquear el video.
100     """
101     app.run(host='0.0.0.0', port=5000, debug=False, threaded=True)
```

1.6.2. Archivo webServer.py

```
1 #!/usr/bin/env python3
2 #
3 # This file is part of Diseño Modular e Incremental para la
4 # Navegación Autónoma Contextual en Sistemas Embebidos de Bajo Coste
5 #
6 # Copyright 2026 Felipe López Castro <i12locaf@uco.es>
7 #
8 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
9 # en Sistemas Embebidos de Bajo Coste is free software: you can
10 # redistribute it
11 # and/or modify it under the terms of the GNU General Public License
12 # as published by the Free Software Foundation, either version 3 of
13 # the License,
14 # or (at your option) any later version.
15 #
16 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
17 # en Sistemas Embebidos de Bajo Coste is distributed in the hope that
18 # it will be useful,
19 # but WITHOUT ANY WARRANTY; without even the implied warranty of
20 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
21 # GNU General Public License for more details.
22 #
23 # You should have received a copy of the GNU General Public License
24 # along with Diseño Modular e Incremental para la Navegación Autónoma
25 # Contextual
26 # en Sistemas Embebidos de Bajo Coste.
27 # If not, see <https://www.gnu.org/licenses/>.
28 #
29 #
30 # Nombre del archivo: webServer.py
31 # Descripción: Servidor Principal del Sistema.
32 #
33 # Integra múltiples servicios Concurrentes:
34 #
35 # 1. Servidor WebSocket (Puerto 8888): Control remoto en
36 # tiempo real y telemetría.
37 #
38 # 2. Servidor Flask (Puerto 5000): Streaming de video
39 # HTTP.
40 #
41 # 3. Lógica de Hardware: Coordinación de motores, servos
42 # y sensores.
43 #
44 import os
45 import asyncio
46 import websockets
47 import json
48 import socket
49 import threading
50 #
51 # Módulos del robot
52 from camera_opencv import Camera # Sistema de cámara
53 import move # Control de motores DC
54 import RPi servo # Control de servos (PCA9685)
55 import functions # Lógica autónoma (PID, Line
56 Following)
57 import robotLight # Control de iluminación RGB
58 import app # Instancia de aplicación Flask
59 #
60 # ===== Estado global del Sistema =====
```

```

49 speed_set = 70          # Velocidad por defecto (0-100)
50 direction_command = 'no' # Estado actual de dirección
51 turn_command = 'no'     # Estado actual de giro
52
53 # Identificadores de canales de Servo
54 SERVO_TILT = 0          # Servo de inclinación vertical de cámara
55 SERVO_PAN = 1           # Servo de paneo horizontal de cámara
56 SERVO_STEERING = 2     # Servo de dirección del vehículo
57
58 # Instancias Globales de Controladores
59 RL = None               # Controlador de Luces
60 fuc = None              # Controlador de Funciones Autónomas
61
62 # ===== Utilidades de Sistema (Optimizado)
63 # =====
64 # Variables estáticas para cálculo diferencial de uso de CPU
65 _prev_cpu_total = None
66 _prev_cpu_idle = None
67
68 def _read_cpu_times():
69     """
70     Lee las estadísticas crudas del kernel desde /proc/stat.
71     Optimización: Evita usar comandos externos como 'top' que son
72     lentos y pesados.
73
74     Retorna:
75     - total: Tiempo total de CPU acumulado.
76     - idle: Tiempo ocioso acumulado (idle + iowait).
77     """
78     try:
79         with open("/proc/stat", "r") as f:
80             line = f.readline()
81             if not line.startswith("cpu "): return None, None
82             parts = list(map(int, line.split()[1:11]))
83             # idle incluye idle normal + iowait (espera de disco)
84             idle_all = parts[3] + parts[4]
85             total = sum(parts)
86             return total, idle_all
87     except: return None, None
88
89 def get_cpu_usage():
90     """
91     Calcula el porcentaje de uso de CPU en tiempo real.
92     Compara los contadores de ciclos entre dos llamadas consecutivas.
93     """
94     global _prev_cpu_total, _prev_cpu_idle
95     t_now, i_now = _read_cpu_times()
96
97     if t_now is None: return "0"
98
99     # Primera lectura: no hay referencia anterior
100     if _prev_cpu_total is None:
101         _prev_cpu_total, _prev_cpu_idle = t_now, i_now
102         return "0"
103
104     delta_total = t_now - _prev_cpu_total
105     delta_idle = i_now - _prev_cpu_idle

```

```

105     # Actualizar estado previo
106     _prev_cpu_total, _prev_cpu_idle = t_now, i_now
107
108     if delta_total == 0: return "0"
109     usage = 100.0 * (1.0 - (delta_idle / delta_total))
110     return f"{usage:.1f}"
111
112 def get_cpu_temp():
113     """Obtiene la temperatura del SoC leyendo la zona térmica del
114     sistema."""
115     try:
116         with open("/sys/class/thermal/thermal_zone0/temp", "r") as f:
117             return round(float(f.read()) / 1000, 2)
118     except: return 0.0
119
120 def get_ram_usage():
121     """Calcula el porcentaje de memoria RAM utilizada leyendo /proc/
122     meminfo."""
123     try:
124         mem = {}
125         with open("/proc/meminfo", "r") as f:
126             for line in f:
127                 parts = line.split(':')
128                 if len(parts) == 2:
129                     mem[parts[0].strip()] = int(parts[1].split()[0])
130
131         total = mem.get('MemTotal', 1)
132         avail = mem.get('MemAvailable', mem.get('MemFree', 0))
133         used_percent = 100 * (1 - (avail / total))
134         return f"{used_percent:.1f}"
135     except: return "0"
136
137 def wifi_check():
138     """
139     Verificación de conectividad de red al inicio.
140     - Si hay internet (ping a 1.1.1.1): pone luces VERDES.
141     - Si no hay red: intenta crear un punto de acceso (AP) y pone
142     luces AZULES.
143     """
144     try:
145         # Intenta conectar a DNS de Cloudflare (sin enviar datos, solo
146         handshake TCP)
147         s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
148         s.settimeout(1.0)
149         s.connect(("1.1.1.1", 80))
150         s.close()
151         print("[Network] Wi-Fi conectado a infraestructura.")
152         if RL: RL.front_color('green')
153     except Exception:
154         print("[Network] Sin conexión. Intentando levantar Hotspot (AP)
155         ...)")
156         if RL: RL.front_color('blue')
157         try:
158             # Script de sistema para modo AP (requiere configuración
159             previa en S0)
160             os.system("sudo create_ap wlan0 eth0 Adeept_Robot &")
161         except: pass

```

```

157 def servoPosInit():
158     """Restablece los servos a su posición segura (Home)."""
159     RPIServo.move(SERVO_STEERING, 88.5) # Centro dirección
160     RPIServo.move(SERVO_TILT, 40)        # Cabeza levemente abajo
161     RPIServo.move(SERVO_PAN, 85)         # Cabeza al frente
162
163 # ===== Lógica de Control del Robot =====
164 def robotCtrl(cmd, ws_obj):
165     """
166     Intérprete de comandos de bajo nivel.
167     Mapea instrucciones textuales a acciones de hardware.
168
169     Args:
170         cmd (str): Comando recibido (ej. 'forward', 'left').
171         ws_obj: Objeto WebSocket (no usado directamente aquí, pero
172         disponible para feedback).
173     """
174     global direction_command, turn_command, speed_set
175
176     # --- Control de Tracción (Motores DC) ---
177     if cmd == 'forward':
178         move.forward(speed_set)
179         if RL: RL.front_color('blue')
180     elif cmd == 'backward':
181         move.backward(speed_set)
182         if RL: RL.front_color('red') # Luz de freno/reversa
183     elif cmd == 'DS': # Drive Stop: Detención inmediata
184         move.motorStop()
185         if RL: RL.front_all_off()
186     elif cmd == 'TS': # Turn Stop: Centrar dirección
187         RPIServo.move(SERVO_STEERING, 88.5)
188
189     # --- Control de Dirección (Servo Eje Delantero) ---
190     elif cmd == 'left':
191         RPIServo.move(SERVO_STEERING, 120) # Límite izquierdo
192         if RL: RL.front_turn_left()
193     elif cmd == 'right':
194         RPIServo.move(SERVO_STEERING, 50)  # Límite derecho
195         if RL: RL.front_turn_right()
196
197     # --- Control de Cámara (Servos Pan/Tilt) ---
198     elif cmd == 'up':
199         RPIServo.move(SERVO_TILT, 110) # Mirar arriba
200     elif cmd == 'down':
201         RPIServo.move(SERVO_TILT, 65)  # Mirar abajo
202     elif cmd == 'home':
203         servoPosInit()
204     elif cmd == 'lookleft':
205         RPIServo.move(SERVO_PAN, 120)
206     elif cmd == 'lookright':
207         RPIServo.move(SERVO_PAN, 50)
208
209 # ===== Lógica del Servidor WebSocket =====
210
211 async def recv_msg(websocket, path):
212     """
213     Bucle principal de comunicación WebSocket por cliente.
214     Recibe, parsea y ejecuta comandos en tiempo real.

```

```

213     """
214     global speed_set
215
216     while True:
217         try:
218             data = await websocket.recv()
219
220             # -----
221             # 1. Comandos de Movimiento Directo
222             # -----
223             if data in ['forward', 'backward', 'DS', 'TS', 'left', 'right',
224                         'lookleft', 'lookright', 'up', 'down', 'home'
225 ]:
226                 robotCtrl(data, websocket)
227
228             # -----
229             # 2. Configuración de Velocidad
230             # -----
231             elif data.startswith('Speed'):
232                 try:
233                     val = int(data.split()[1])
234                     speed_set = move.speed_set(val) # Actualiza valor
235 y retorna el limitado
236                 except: pass
237
238             # -----
239             # 3. Modos Autónomos (Computer Vision)
240             # -----
241             elif data == 'trackLine':
242                 # Activa seguimiento de línea
243                 if RL: RL.front_color('white')
244                 fuc.modeSet('trackLine')
245
246             elif data == 'findColor':
247                 # Activa búsqueda de color (Feature legacy)
248                 if RL: RL.front_color('yellow')
249                 Camera().modeselect('findColor')
250
251             elif data == 'stopCV' or data == 'pauseFunctions':
252                 # Detiene cualquier modo autónomo
253                 if RL: RL.front_color('black') # Luces apagadas
254                 fuc.pause()
255                 Camera().modeselect('none')
256
257             # -----
258             # 4. Telemetría de Sistema
259             # -----
260             elif data == 'get_info':
261                 # Responde con un JSON conteniendo estado de salud del
262 sistema
263                 info = json.dumps({
264                     'title': 'info_update',
265                     'data': {
266                         'CPU_Temp': get_cpu_temp(), # C
267                         'CPU_Usage': get_cpu_usage(), # %
268                         'RAM_Usage': get_ram_usage() # %
269                     }
270                 })

```



```

267         })
268         await websocket.send(info)
269
270         # -----
271         # 5. Mantenimiento de Conexión
272         # -----
273         elif data == 'ping':
274             pass # Keep-alive
275
276         except websockets.exceptions.ConnectionClosed:
277             print("[WebSocket] Cliente desconectado (ConnectionClosed)
278 ")
279             move.motorStop() # Seguridad: Detener robot si se pierde
280 control
281             break
282         except Exception as e:
283             print(f"[WebSocket] Error de protocolo: {e}")
284             break
285
286 async def main_logic(websocket, path):
287     """
288     Handshake inicial de conexión WebSocket.
289     Implementa autenticación básica antes de permitir control.
290     """
291     try:
292         # Espera credenciales como primer mensaje
293         auth = await websocket.recv()
294         # Credenciales harcodeadas (Por defecto de fábrica)
295         if auth.strip() == "admin:123456":
296             await websocket.send("congratulation")
297             await recv_msg(websocket, path) # Transferir a bucle de
298 comandos
299         else:
300             await websocket.send("sorry")
301     except: pass
302
303 # ===== Punto de Entrada Principal =====
304 if __name__ == '__main__':
305     print("[Init] Inicializando sistema Adeept PiCar-B...")
306
307     # 1. Inicialización de Hardware Base (Motores/Servos)
308     move.setup()
309     RPIservo.init() if hasattr(RPIservo, 'init') else None
310     servoPosInit()
311
312     # 2. Inicialización de Subsistema de Iluminación
313     try:
314         RL = robotLight.RobotLight()
315     except Exception as e:
316         print(f"[Init] Error iniciando luces: {e}")
317         RL = None
318
319     # 3. Inicialización de Controlador Autónomo (Hilo independiente)
320     fuc = functions.Functions()
321     fuc.start()
322
323     # 4. Inicialización de Cámara (Singleton)
324     # Nota: Instanciar Camera() configura el hardware, los hilos de

```

```

captura se inician internamente.
322     try:
323         cam = Camera()
324         print("[Init] Cámara iniciada y lista.")
325     except Exception as e:
326         print(f"[Init] Error crítico en cámara: {e}")
327
328     # 5. Servidor de Streaming de Video (Flask)
329     # Se ejecuta en un hilo Demonio para no bloquear el Event Loop de
    asyncio.
330     try:
331         flask_thread = threading.Thread(target=lambda: app.app.run(
    host='0.0.0.0', port=5000, debug=False, use_reloader=False))
332         flask_thread.daemon = True
333         flask_thread.start()
334         print("[Init] Servidor Video (Flask) escuchando en puerto 5000
    ")
335     except Exception as e:
336         print(f"[Init] Error iniciando Flask: {e}")
337
338     # 6. Comprobación de Red
339     wifi_check()
340
341     # 7. Servidor de Control (WebSocket)
342     print("[Init] Servidor Control (WebSocket) iniciando en puerto
    8888...")
343
344     # Configuración del Event Loop de Asyncio
345     loop = asyncio.new_event_loop()
346     asyncio.set_event_loop(loop)
347     start_server = websockets.serve(main_logic, '0.0.0.0', 8888)
348
349     try:
350         loop.run_until_complete(start_server)
351         print("[System] Sistema en línea. Esperando conexiones.")
352         loop.run_forever()
353     except KeyboardInterrupt:
354         print("\n[System] Apagando sistema ordenadamente...")
355     finally:
356         # Limpieza de recursos al salir (CTRL+C)
357         move.motorStop()
358         move.destroy()
359         if RL: RL.cleanup()

```

1.6.3. Archivo GUI.py (cliente)

```
1 #!/usr/bin/python3
2 #
3 # This file is part of Diseño Modular e Incremental para la
4 # Navegación Autónoma Contextual en Sistemas Embebidos de Bajo Coste
5 #
6 # Copyright 2026 Felipe López Castro <i12locaf@uco.es>
7 #
8 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
9 # en Sistemas Embebidos de Bajo Coste is free software: you can
10 # redistribute it
11 # and/or modify it under the terms of the GNU General Public License
12 # as published by the Free Software Foundation, either version 3 of
13 # the License,
14 # or (at your option) any later version.
15 #
16 # Diseño Modular e Incremental para la Navegación Autónoma Contextual
17 # en Sistemas Embebidos de Bajo Coste is distributed in the hope that
18 # it will be useful,
19 # but WITHOUT ANY WARRANTY; without even the implied warranty of
20 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
21 # GNU General Public License for more details.
22 #
23 # You should have received a copy of the GNU General Public License
24 # along with Diseño Modular e Incremental para la Navegación Autónoma
25 # Contextual
26 # en Sistemas Embebidos de Bajo Coste.
27 # If not, see <https://www.gnu.org/licenses/>.
28 #
29 # -*- coding: UTF-8 -*-
30 # File name : GUI.py
31 # Description : Cliente Gráfico (PC Host) para control remoto del
32 # PiCar-B.
33 #
34 # Desarrollado con Tkinter (Interfaz) y OpenCV (
35 # Visualización).
36 # Características:
37 # - Comunicación bidireccional vía WebSocket (Control +
38 # Telemetría).
39 # - Streaming de video MJPEG desde el servidor Flask.
40 # - Controles manuales (Teclado/Botones) para Motores y
41 # Cámara.
42 # - Herramientas de depuración de Visión (Ajuste HSV en
43 # tiempo real).
44 #
45 import time
46 import threading
47 import tkinter as tk
48 import asyncio
49 import websockets.protocol
50 import json
51 import cv2
52 import numpy as np
53
54 # --- Constantes de Estilo (Tema Oscuro) ---
55 color_bg='#000000' # Fondo negro
56 color_text='#E1F5FE' # Texto claro (Azul muy pálido)
```

```

48 color_btn='#0277BD'          # Botones azules (Material Design)
49
50 # ===== Estado Global del Cliente =====
51
52 # Gestión de botones activos (exclusión mutua para modos autónomos)
53 function_button_active = None
54
55 # Estados de control manual (para evitar reenvío de comandos repetidos
56 )
57 steering_state = 'center'
58 camera_pan_state = 'stop'
59 camera_tilt_state = 'stop'
60 ip_stu = 1 # Estado de la conexión (1=Desconectado, 0=Conectado)
61
62 # Variables Reactivas de Tkinter (Data Binding)
63 cpu_temp_var = None
64 cpu_use_var = None
65 ram_use_var = None
66 var_Speed = None
67
68 # Control del hilo de visualización de video
69 video_thread = None
70 video_stop_event = None
71
72 # --- Estado del Modo de Calibración HSV (Visión) ---
73 # Permite ajustar los rangos de color desde el cliente y ver la má
74   scara resultante.
75 hsv_mode = False
76 hsv_windows_created = False
77 hsv_init = {"H_min": 0, "S_min": 0, "V_min": 0, "H_max": 179, "S_max":
78   255, "V_max": 255}
79
80 # --- Infraestructura de Red ---
81 websocket = None
82 event_loop = None
83
84 def reset_client_state():
85     """Restablece los estados internos de la interfaz al conectar/
86     desconectar."""
87     global function_button_active, steering_state, camera_pan_state,
88     camera_tilt_state
89     global hsv_mode, hsv_windows_created
90
91     function_button_active = None
92     steering_state = 'center'
93     camera_pan_state = 'stop'
94     camera_tilt_state = 'stop'
95
96     # Reiniciar herramientas de visión
97     hsv_mode = False
98     if hsv_windows_created:
99         _hsv_destroy_windows()
100
101     print("[GUI] Estados reiniciados")
102
103 # ===== Herramientas de Depuración de Visión (HSV)
104   =====

```

```

100 def _hsv_create_windows():
101     """Crea ventanas de OpenCV con Sliders (Trackbars) para calibración de color."""
102     global hsv_windows_created
103     if hsv_windows_created: return
104     try:
105         cv2.namedWindow("Controls")
106         cv2.resizeWindow("Controls", 500, 300)
107         # Sliders para rango inferior (Min)
108         cv2.createTrackbar("H_min", "Controls", hsv_init["H_min"],
179, lambda x: None)
109         cv2.createTrackbar("S_min", "Controls", hsv_init["S_min"],
255, lambda x: None)
110         cv2.createTrackbar("V_min", "Controls", hsv_init["V_min"],
255, lambda x: None)
111         # Sliders para rango superior (Max)
112         cv2.createTrackbar("H_max", "Controls", hsv_init["H_max"],
179, lambda x: None)
113         cv2.createTrackbar("S_max", "Controls", hsv_init["S_max"],
255, lambda x: None)
114         cv2.createTrackbar("V_max", "Controls", hsv_init["V_max"],
255, lambda x: None)
115         hsv_windows_created = True
116     except Exception as e:
117         print(f"[HSV] Error creando ventanas: {e}")
118
119 def _hsv_destroy_windows():
120     """Cierra las ventanas de depuración."""
121     global hsv_windows_created
122     if not hsv_windows_created: return
123     try:
124         cv2.destroyWindow("Controls")
125         cv2.destroyWindow("Mask")
126     except: pass
127     hsv_windows_created = False
128
129 def _hsv_get_range():
130     """Lee los valores actuales de los sliders."""
131     if not hsv_windows_created: return (0,0,0,179,255,255)
132     return (cv2.getTrackbarPos("H_min", "Controls"), cv2.
getTrackbarPos("S_min", "Controls"),
133             cv2.getTrackbarPos("V_min", "Controls"), cv2.
getTrackbarPos("H_max", "Controls"),
134             cv2.getTrackbarPos("S_max", "Controls"), cv2.
getTrackbarPos("V_max", "Controls"))
135
136 def _hsv_enable(flag: bool):
137     """Activa el modo de visualización de máscara binaria."""
138     global hsv_mode
139     hsv_mode = bool(flag)
140
141 # ===== Comunicación WebSocket =====
142
143 def send_command(command):
144     """Envía un comando de texto al servidor robot."""
145     if websocket and websocket.state == websockets.protocol.State.OPEN
:
146         asyncio.run_coroutine_threadsafe(websocket.send(str(command)),

```

```

event_loop)
147
148 # ===== Streaming de Video =====
149
150 def show_video_stream(ip_address, stop_event):
151     """
152     Hilo dedicado a la captura y visualización del stream MJPEG.
153     Se conecta al servidor Flask (puerto 5000) y muestra frames con
154     OpenCV.
155     """
156     video_url = f"http://{ip_address}:5000/video_feed"
157     window_name = "Stream"
158     window_created = False
159     cap = None
160
161     while not stop_event.is_set():
162         # Lógica de reconexión automática
163         if cap is None or not cap.isOpened():
164             try:
165                 if cap: cap.release()
166                 cap = cv2.VideoCapture(video_url)
167             except: cap = None
168             time.sleep(0.5)
169             continue
170
171         ret, frame = cap.read()
172         if not ret:
173             time.sleep(0.2); cap.release(); cap = None
174             continue
175
176         if not window_created:
177             cv2.namedWindow(window_name, cv2.WINDOW_AUTOSIZE)
178             window_created = True
179
180         # Procesamiento de imagen en cliente (solo visualización)
181         if hsv_mode:
182             if not hsv_windows_created: _hsv_create_windows()
183             hm, sm, vm, hM, sM, vM = _hsv_get_range()
184             hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
185             mask = cv2.inRange(hsv, np.array([hm, sm, vm]), np.array([
186             hM, sM, vM]))
187             cv2.imshow("Mask", mask)
188         else:
189             if hsv_windows_created: _hsv_destroy_windows()
190
191             cv2.imshow(window_name, frame)
192             if cv2.waitKey(1) & 0xFF == 27: # Tecla ESC para salir
193                 stop_event.set(); break
194
195         if cap: cap.release()
196         try: cv2.destroyAllWindows()
197         except: pass
198         _hsv_destroy_windows()
199
200 # ===== Lógica de Red y Telemetría =====
201
202 def request_system_info():
203     """Solicita periódicamente (Polling) información del sistema al

```

```

servidor."""
202     if websocket:
203         send_command('get_info')
204         # Programar próxima llamada en 2000ms (2 segundos)
205         root.after(2000, request_system_info)
206
207 async def network_loop(ip_address, port):
208     """
209     Bucle principal asíncrono de red.
210     Maneja la conexión WebSocket y la recepción de mensajes.
211     """
212     global websocket, event_loop, ip_stu
213     event_loop = asyncio.get_running_loop()
214     uri = f"ws://{ip_address}:{port}"
215     try:
216         async with websockets.connect(uri) as ws:
217             websocket = ws
218             print("Conectado al servidor!")
219             reset_client_state()
220
221             # Actualización de UI
222             l_ip_4.config(text='Connected', bg='#558B2F'); l_ip_5.
config(text=f'IP:{ip_address}')
223             E1.config(state='disabled'); Btn_Connect.config(state='
disabled'); ip_stu=0
224
225             # Persistencia de última IP
226             with open("last_ip.txt", "w") as f: f.write(ip_address)
227
228             # Iniciar hilo de video
229             global video_thread, video_stop_event
230             video_stop_event = threading.Event()
231             video_thread = threading.Thread(target=show_video_stream,
args=(ip_address, video_stop_event), daemon=True)
232             video_thread.start()
233
234             # Autenticación
235             await websocket.send("admin:123456")
236             request_system_info()
237
238             # Bucle de recepción de mensajes
239             while True:
240                 message = await websocket.recv()
241                 try:
242                     data = json.loads(message)
243                     # Procesamiento de Telemetría
244                     if data.get('title') == 'info_update':
245                         info = data.get('data', {})
246                         temp = info.get('CPU_Temp') or info.get('
cpu_temp', 'N/A')
247                         cpu = info.get('CPU_Usage') or info.get('
cpu_usage', info.get('cpu_use', 'N/A'))
248                         ram = info.get('RAM_Usage') or info.get('
ram_usage', info.get('ram_use', 'N/A'))
249
250                         cpu_temp_var.set(f"{temp}")
251                         cpu_use_var.set(f"{cpu}")
252                         ram_use_var.set(f"{ram}")

```

```

253         except Exception as e:
254             print(f"Error procesando mensaje: {e}")
255
256     except Exception as e:
257         print(f"Error de conexión: {e}")
258     finally:
259         # Limpieza al desconectar
260         websocket = None
261         if video_stop_event: video_stop_event.set()
262         ip_stu = 1
263         E1.config(state='normal'); Btn_Connect.config(state='normal')
264         l_ip_4.config(text='Disconnected', bg='#F44336'); l_ip_5.
265         config(text='<No IP>')
266         cpu_temp_var.set("N/A"); cpu_use_var.set("N/A"); ram_use_var.
267         set("N/A")
268
269 def start_network_thread():
270     """Lanzador del hilo de red desde el botón Conectar."""
271     ip_adr = E1.get().strip()
272     if not ip_adr: return
273     l_ip_4.config(text='Connecting', bg='#FF8F00')
274     threading.Thread(target=lambda: asyncio.run(network_loop(ip_adr,
275     8888)), daemon=True).start()
276
277 def connect(event):
278     if ip_stu == 1: start_network_thread()
279
280 # ===== Callback de Controles UI =====
281
282 def speed_send(event):
283     """Envía la actualización de velocidad cuando se mueve el slider.
284     """
285     send_command(f'Speed {var_Speed.get()}')
286
287 # --- Control de Cámara (Pan/Tilt) ---
288
289 def call_up(event):
290     global camera_tilt_state
291     if camera_tilt_state != 'up': send_command('up');
292     camera_tilt_state = 'up'
293     else: send_command('home'); camera_tilt_state = 'stop'
294
295 def call_down(event):
296     global camera_tilt_state
297     if camera_tilt_state != 'down': send_command('down');
298     camera_tilt_state = 'down'
299     else: send_command('home'); camera_tilt_state = 'stop'
300
301 def call_lookleft(event):
302     global camera_pan_state
303     if camera_pan_state != 'left': send_command('lookleft');
304     camera_pan_state = 'left'
305     else: send_command('home'); camera_pan_state = 'stop'
306
307 def call_lookright(event):
308     global camera_pan_state
309     if camera_pan_state != 'right': send_command('lookright');
310     camera_pan_state = 'right'

```



```

303     else: send_command('home'); camera_pan_state = 'stop'
304
305 def call_home(event):
306     """Centra la cámara y detiene servos."""
307     global camera_pan_state, camera_tilt_state
308     send_command('home'); camera_pan_state = 'stop'; camera_tilt_state
309     = 'stop'
310
311 # --- Control de Dirección y Tracción ---
312 def call_turn_left(event):
313     global steering_state
314     if steering_state != 'left': send_command('left'); steering_state
315     = 'left'
316     else: send_command('TS'); steering_state = 'center'
317
318 def call_turn_right(event):
319     global steering_state
320     if steering_state != 'right': send_command('right');
321     steering_state = 'right'
322     else: send_command('TS'); steering_state = 'center'
323
324 def call_forward(event): send_command('forward')
325 def call_backward(event): send_command('backward')
326 def call_stop(event): send_command('DS') # Drive Stop
327
328 # --- Gestión de Botones de Modo (Toggle) ---
329 def toggle_function(button_widget, start_command, stop_command):
330     """Maneja el estado activo/inactivo de los botones de función."""
331     global function_button_active, motor_controls, servo_controls
332
333     # Desactivar botón previo visualmente
334     if function_button_active and function_button_active !=
335     button_widget:
336         function_button_active.config(bg=color_btn)
337
338     # Si se pulsa el mismo botón activo -> Apagar
339     if function_button_active == button_widget:
340         send_command(stop_command)
341         button_widget.config(bg=color_btn)
342         function_button_active = None
343         # Rehabilitar controles manuales
344         for btn in motor_controls + servo_controls: btn.config(state='
345         normal')
346
347     # Si se pulsa un botón nuevo -> Encender
348     else:
349         send_command(start_command)
350         button_widget.config(bg='#4CAF50') # Verde activo
351         function_button_active = button_widget
352         # Deshabilitar controles manuales en modos automáticos (salvo
353         FindColor)
354         disable_controls = (start_command != 'findColor')
355         for btn in motor_controls + servo_controls:
356             btn.config(state='disabled' if disable_controls else '
357             normal')
358
359 def call_FindColor(event):
360     toggle_function(Btn_FindColor, 'findColor', 'stopCV')

```

```

354     _hsv_enable(function_button_active == Btn_FindColor)
355
356 def call_LineTrack(event):
357     toggle_function(Btn_LineTrack, 'trackLine', 'pauseFunctions')
358
359
360 # ===== Construcción de Interfaz Gráfica (Tkinter)
361     =====
362 def loop():
363     global root, cpu_temp_var, cpu_use_var, ram_use_var, var_Speed
364     global E1, Btn_Connect, l_ip_4, l_ip_5
365
366     root = tk.Tk(); root.title('Control');
367     root.geometry('720x300'); # Dimensiones de ventana
368     root.config(bg=color_bg)
369
370     # Variables vinculadas a etiquetas de texto
371     cpu_temp_var = tk.StringVar(value="N/A")
372     cpu_use_var = tk.StringVar(value="N/A")
373     ram_use_var = tk.StringVar(value="N/A")
374     var_Speed = tk.StringVar(); var_Speed.set(100)
375
376     try:
377         logo=tk.PhotoImage(file='logo.png'); tk.Label(root,image=logo,
378         bg=color_bg).place(x=30,y=13)
379     except: pass
380
381     # --- Posicionamiento de Componentes ---
382
383     # 1. Panel Superior: Conexión IP y Telemetría
384     connent_input(125, 15)
385     information_screen(380, 15)
386
387     # 2. Panel Derecho: Botones de Función Autónoma
388     function_buttons(350, 205)
389
390     # 3. Panel Inferior Izquierdo: Controles de Motor
391     motor_buttons(30, 115)
392
393     # 4. Panel Inferior Derecho: Controles de Servo/Cámara
394     servo_buttons(350, 115)
395
396     # 5. Slider de Velocidad
397     scale_speed(30, 230, 203)
398
399     # Auto-rellenar última IP usada
400     try:
401         with open("last_ip.txt") as f:
402             last = f.read().strip()
403             if last: E1.insert(0, last)
404     except: pass
405
406     root.mainloop()
407
408 def connent_input(x,y):
409     global E1, Btn_Connect
410     tk.Label(root,width=10,text='IP Address:',fg=color_text,bg='
#000000').place(x=x,y=y)

```

```

409     E1 = tk.Entry(root, show=None, width=16, bg="#37474F", fg='#e6eff1');
E1.place(x=x+5, y=y+25)
410     Btn_Connect= tk.Button(root, width=8, height=2, text='Connect', fg=
color_text, bg=color_btn, relief='ridge')
411     Btn_Connect.place(x=x+130, y=y)
412     root.bind('<Return>', connect); Btn_Connect.bind('<ButtonPress-1>',
, connect)
413
414 def information_screen(x, y):
415     global l_ip_4, l_ip_5
416     # Etiquetas estáticas
417     tk.Label(root, width=10, text='CPU Temp:', fg=color_text, bg=color_bg,
anchor='w').place(x=x, y=y)
418     tk.Label(root, width=10, text='CPU Usage:', fg=color_text, bg=color_bg
, anchor='w').place(x=x, y=y+30)
419     tk.Label(root, width=10, text='RAM Usage:', fg=color_text, bg=color_bg
, anchor='w').place(x=x, y=y+60)
420
421     # Valores dinámicos
422     tk.Label(root, width=8, textvariable=cpu_temp_var, fg=color_text, bg=
color_bg, anchor='w').place(x=x+80, y=y)
423     tk.Label(root, width=8, textvariable=cpu_use_var, fg=color_text, bg=
color_bg, anchor='w').place(x=x+80, y=y+30)
424     tk.Label(root, width=8, textvariable=ram_use_var, fg=color_text, bg=
color_bg, anchor='w').place(x=x+80, y=y+60)
425
426     # Unidades
427     tk.Label(root, width=10, text=' C', fg=color_text, bg=color_bg, anchor
='w').place(x=x+110, y=y)
428     tk.Label(root, width=10, text=' %', fg=color_text, bg=color_bg, anchor
='w').place(x=x+110, y=y+30)
429     tk.Label(root, width=10, text=' %', fg=color_text, bg=color_bg, anchor
='w').place(x=x+110, y=y+60)
430
431     # Estado de conexión
432     l_ip_4=tk.Label(root, width=18, text='Disconnected', fg=color_text, bg
='#F44336'); l_ip_4.place(x=x+180, y=y)
433     l_ip_5=tk.Label(root, width=18, text='<No IP>', fg=color_text, bg=
color_btn); l_ip_5.place(x=x+180, y=y+35)
434
435 def motor_buttons(x, y):
436     global motor_controls
437     motor_controls = []
438
439     # Definición de botones de movimiento (WASD compatible)
440     btn_fwd = tk.Button(root, width=8, text='Forward', fg=color_text, bg
=color_btn, relief='ridge')
441     btn_fwd.place(x=x+90, y=y)
442     btn_fwd.bind('<ButtonPress-1>', call_forward); btn_fwd.bind('<
ButtonRelease-1>', call_stop)
443     root.bind('<KeyPress-w>', call_forward); root.bind('<KeyRelease-w>
', call_stop)
444     motor_controls.append(btn_fwd)
445
446     btn_bwd = tk.Button(root, width=10, text='Backward', fg=color_text,
bg=color_btn, relief='ridge')
447     btn_bwd.place(x=x+90, y=y+35)
448     btn_bwd.bind('<ButtonPress-1>', call_backward); btn_bwd.bind('<

```

```

449     ButtonRelease-1>', call_stop)
450     root.bind('<KeyPress-s>', call_backward); root.bind('<KeyRelease-s>', call_stop)
451     motor_controls.append(btn_bwd)
452     btn_left = tk.Button(root, width=8, text='Left',fg=color_text,bg=
453     color_btn,relief='ridge')
454     btn_left.place(x=x,y=y+35)
455     btn_left.bind('<ButtonPress-1>', call_turn_left); root.bind('<
456     KeyPress-a>', call_turn_left)
457     motor_controls.append(btn_left)
458     btn_right = tk.Button(root, width=8, text='Right',fg=color_text,bg=
459     color_btn,relief='ridge')
460     btn_right.place(x=x+180,y=y+35)
461     btn_right.bind('<ButtonPress-1>', call_turn_right); root.bind('<
462     KeyPress-d>', call_turn_right)
463     motor_controls.append(btn_right)
464
465 def servo_buttons(x,y):
466     global servo_controls
467     servo_controls = []
468
469     # Definición de botones de cámara (IJKL compatible)
470     btn_up = tk.Button(root, width=8, text='Up',fg=color_text,bg=
471     color_btn,relief='ridge')
472     btn_up.place(x=x+90,y=y)
473     btn_up.bind('<ButtonPress-1>', call_up); root.bind('<KeyPress-i>',
474     call_up)
475     servo_controls.append(btn_up)
476
477     btn_down = tk.Button(root, width=8, text='Down',fg=color_text,bg=
478     color_btn,relief='ridge')
479     btn_down.place(x=x+90,y=y+35)
480     btn_down.bind('<ButtonPress-1>', call_down); root.bind('<KeyPress-
481     k>', call_down)
482     servo_controls.append(btn_down)
483
484     btn_left = tk.Button(root, width=8, text='Left',fg=color_text,bg=
485     color_btn,relief='ridge')
486     btn_left.place(x=x,y=y+35)
487     btn_left.bind('<ButtonPress-1>', call_lookleft); root.bind('<
488     KeyPress-j>', call_lookleft)
489     servo_controls.append(btn_left)
490
491     btn_right = tk.Button(root, width=8, text='Right',fg=color_text,bg=
492     color_btn,relief='ridge')
493     btn_right.place(x=x+180,y=y+35)
494     btn_right.bind('<ButtonPress-1>', call_lookright); root.bind('<
495     KeyPress-l>', call_lookright)
496     servo_controls.append(btn_right)
497
498     btn_center = tk.Button(root, width=8, text='Center',fg=color_text,
499     bg=color_btn,relief='ridge')
500     btn_center.place(x=x+180,y=y)
501     btn_center.bind('<ButtonPress-1>', call_home); root.bind('<
502     KeyPress-h>', call_home)

```

```

491 def scale_speed(x,y,w):
492     tk.Scale(root,label=None,from_=0,to=100,orient=tk.HORIZONTAL,
493             length=w,showvalue=1,
494             tickinterval=None,resolution=1,variable=var_Speed,
495             troughcolor='#448AFF',
496             command=speed_send,fg=color_text,bg=color_bg,
497             highlightthickness=0).place(x=x,y=y)
498
499 def function_buttons(x,y):
500     global Btn_FindColor, Btn_LineTrack
501
502     # Botón de modo de calibración de color
503     Btn_FindColor = tk.Button(root, width=12, text='Find Color',fg=
504     color_text,bg=color_btn,relief='ridge')
505     Btn_FindColor.place(x=x,y=y)
506     Btn_FindColor.bind('<ButtonPress-1>', call_FindColor)
507
508     # Botón de seguidor de línea autónomo
509     Btn_LineTrack = tk.Button(root, width=12, text='Line Track',fg=
510     color_text,bg=color_btn,relief='ridge')
511     Btn_LineTrack.place(x=x,y=y+35)
512     Btn_LineTrack.bind('<ButtonPress-1>', call_LineTrack)
513
514 if __name__ == '__main__':
515     loop()

```